



# DIGITAL DESIGN AND COMPUTER ORGANIZATION

## Basic Structure of computers

---

T2: Chapter 1: 1.1-1.4

Department of Computer Science and Engineering

# DIGITAL DESIGN AND COMPUTER

## ORGANIZATION Computer Types T2:Ch1 1.1

Department of Computer Science and Engineering

# Basic Structure of computers

## Outline

---

- Computer Types
- Functional Units:
  - Input Unit,
  - Memory Unit,
  - ALU,
  - Output Unit,
  - Control Unit,
- Basic operational concepts

# Basic Structure of computers

## Introduction T2- section 1.1

---

- What is Computer Organization ?

**Computer Organization:** This field focuses on the **functional units of a computer system and how they work together to execute instructions**. It deals with the **internal structure and operation of a computer**, including its components like the CPU, memory, I/O devices, and control unit.

- How is Computer Organization Different from Digital Design ?

While computer organization deals with the **high-level structure** and function of a computer, digital design focuses on the **low-level implementation** of its components.

# Basic Structure of computers

## What is a computer ?

---

- Simply put, a computer is a sophisticated electronic calculating machine that:
  - Accepts **input** information,
  - **Processes** the information according to a list of internally stored instructions and
  - Produces the resulting **output** information.
- Functions performed by a computer are:
  - Accepting information to be processed as input.
  - Storing a list of instructions to process the information.
  - Processing the information according to the list of instructions.
  - Providing the results of the processing as output.

*What are the functional units of a computer?*

# Basic Structure of computers

## Types Of Computers

---

### Classification #1:

- Micro Computers
- Mini Computers
- Mainframes
- Super Computers

### Classification #2:

- Analog Computers
- Digital Computers
- Hybrid Computers

### More General Classification:

- *General Purpose Computers*
- *Special Purpose Computers*

- **Workstations** are *industry* – standard desktop computers with more computational power and high-resolution graphic display with variety of graphic input/output capability.

### What is the use of a workstation ?

- Workstation PC's are usually employed in the applications of Computer Aided Design and Drafting (**CADD**). **Simulation** & Modeling, Interactive Graphic design, Multimedia and other engineering applications.

# Types Of Computers

## Enterprise Systems or Mainframes

- Enterprise systems or mainframes are like a family of computers with tremendous computing power beyond workstations.
- Their computational activities are distributed among one main system (**usually a server**) and number of child nodes or intelligent PC's with or without local computing power / processor (usually **client computers**) called dumb terminals.
- Mainframes are preferred at business data processing corporate offices. They are quite expensive with several hard disks, RAID's and backup storage units.



# Types Of Computers

## Super Computers

- One of the fastest computer's type currently available, *They are **problem scalable**.*
- Computational speed of a super computer is measured in terms of Floating Point Operations Per Second or **FLOPS**.
- Examples: Cray X/MP-14 , Param – 8000, Param - Padma



# Basic Structure of computers

## Functional Units T2:Ch1 1.2

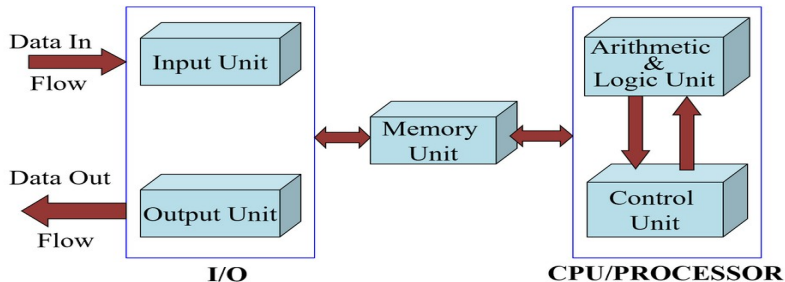
A computer in its simplest form comprises five functional units:

- 1) Input Unit
- 2) Output Unit
- 3) Memory Unit
- 4) Arithmetic & Logic Unit
- 5) Control Unit

# Basic Structure of computers

## Functional Units

The following figure depicts functional units of a computer:



**Stored program concept**-Serially fetching and executing machine code instructions stored in main memory by a processor that performs arithmetic and logical operations

Language translators- compilers

Information must be encoded -encodings- ASCII,EBCDIC

# Functional Units of a Computer

## Input Unit

- Computer accepts encoded information through input unit. The standard input device is a keyboard of a video monitor or terminal.
- Whenever a key is pressed, keyboard controller sends the scanned code of that letter, digit or symbol to CPU/Memory.
- Examples include Mouse, Joystick, Tablet or Digitizer, Scanner etc.



Memory unit stores the program **instructions**, **data** operands on and results of computations etc.

Memory has large semi-conductor storage cells.

- Words
- Word length
- Memory access time- time required to access one word
- RAM, cache, mainmemory(slowest and largest)
- Memory unit is classified as:
  - **1. Primary /Main Memory**
  - **2. Secondary /Auxiliary Memory**

**Primary memory** is the computer memory that is directly accessible by CPU. It is comprised of **DRAM** and provides the actual working space to the processor. It holds the data and instructions that the processor is currently working on.

- **Advantages of Primary Memory**
  - a. **Speed:** Provides fast access to data and instructions.
  - b. **Direct Access:** Allows the CPU to quickly read from and write to memory.
- **Disadvantages of Primary Memory**
  - a. **Limited Size:** Is smaller compared to secondary memory

# Functional Units of a Computer

## Memory Unit

---

### Secondary Memory / Mass Storage

The contents of the secondary memory first get transferred to the primary memory and then are accessed by the processor, **this is because the processor does not directly interact with the secondary memory.**

- **Advantages of Secondary Memory**
  - **Persistence:** Retains data even without power.
  - **Large Capacity:** Typically offers much more storage space than primary memory.
- **Disadvantages of Secondary Memory**
  - **Speed:** Slower access compared to primary memory.

# Functional Units of a Computer

## Memory Unit

1. Main memory is classified again as RAM and ROM.
  - **RAM** is termed as Read/Write memory or user memory that holds run time program instruction and data.
  - **ROM** holds system programs and firmware routines such as BIOS, POST, I/O Drivers that are essential to manage the hardware of a computer.



RAM



ROM



# Functional Units of a Computer

## Memory Unit

---

The key difference between RAM and ROM lies in their:

### **Purpose:**

RAM: Temporary storage for data and program instructions, lost when power is turned off.

ROM: Permanent storage for firmware or system software.

### **Write capability:**

RAM: Read-write (data can be written, modified, and erased).

ROM: Read-only (data cannot be modified after manufacture).

### **Volatility:**

RAM: Volatile (loses data when power is turned off).

ROM: Non-volatile (retains data even without power).

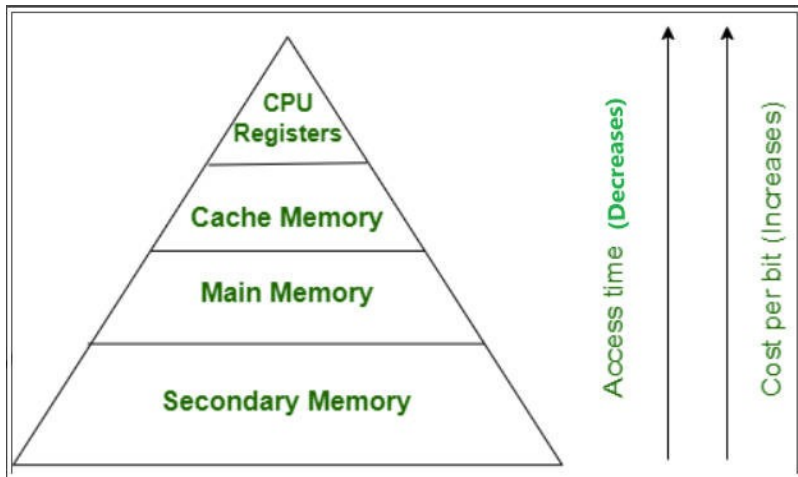
### 2. Secondary /Auxiliary Memory:

- While primary storage is essential, it is **volatile** in nature (i.e. its contents will be lost in the absence of power) and expensive too. Additional requirement of memory would be supplied as auxiliary memory at **cheaper cost**.
- Secondary memory are *magnetic memories* viz Floppy disk, Hard disk, Magnetic tape, CD-ROM etc.
- Secondary memories are **non volatile** in nature (contents will not be lost in the absence of power).



# Functional Units of a Computer

## Memory Hierarchy



### What is an ALU?

- An ALU is a digital circuit that performs **arithmetic and logic operations** on binary numbers.
- ALUs are essential components of computer systems, as they enable the execution of various computational tasks.

### How does an ALU work?

- ALUs **process binary data using logic gates**, which perform operations like AND, OR, NOT, and XOR. The combination of these gates allows ALUs to execute complex arithmetic and logic operations.

# Functional Units of a Computer

## Functions of an ALU

- Operations are **executed** in the Arithmetic and Logic Unit (ALU).
  - Arithmetic operations such as addition, subtraction.
  - Logic operations such as comparison of numbers.
- In order **to execute** an instruction, **operands** need to be brought into the ALU **from the memory**.
  - Operands are stored **in general purpose registers** available in the ALU.
  - Access times of general purpose registers are faster than the cache.
- **Results** of the operations are **stored back** in the memory or retained in the processor for immediate use.

# Functional Units of a Computer

## Output Unit

---

- Computer **returns** the computed results, error messages, etc., via **output unit**.
- The standard output device is a video monitor, LCD/TFT monitor.
- Other output devices are *printers*, *plotters* to take a paper copy of the results, programs, graphs called *print out* or a hardcopy
- Printers types: Dot Matrix printer, Inkjet printer, Laser printer. . .

# Functional Units of a Computer

## Control Unit

- **Coordinates activities:** The control unit oversees the operations of all components within the CPU.
- **Issues timing signals:** It sends signals like **MEMR** (Memory Read), **MEMW** (Memory Write), **IOR** (Input Output Read), and **IOW** (Input Output Write) to control data transfers.
- **Governs data transfers:** These timing signals determine **when specific operations should occur**.
- **Interprets instructions:** The control unit decodes instructions to determine the required actions.

# Functional Units of a Computer

## Control Unit

---

### Operation of a computer can be summarized as:

- **Accepts** information from the input units (**Input unit**).
- **Stores** the information (**Memory**).
- **Processes** the information (**ALU**).
- Provides **processed results** through the output units (**Output unit**).
- **Operations** of Input unit, Memory, ALU and Output unit are **coordinated** by **Control unit**.
  - **Instructions control “what”** operations take place (e.g. data transfer, processing).
  - **Control unit** generates timing signals which determines **“when”** a particular operation takes place.



## Main conceptual events/operations of a computer

1. A set of instructions which perform a given task, called a ***program*** must reside in the main memory of computer during its execution.
2. The **CPU** fetches those instructions sequentially one-by-one from the main memory, decodes them and perform the specified operation on associated data operands in ALU.
3. **Processed data** i.e. useful information will be displayed on an output unit.
4. All activities pertaining to processing and data movement inside the computer machine are governed by **control unit**.

Information Handled by a Computer:

## Instructions/machine instructions

- Govern the transfer of information within a computer as well as between the computer and its I/O devices
- Specify the arithmetic and logic operations to be performed
- List of instructions is called a Program(in memory)

## Data (numbers or encoded chars)

- Used as operands by the instructions. Eg(Binary, BCD, ASCII,EBCDIC)

# Functional Units of a Computer

Information in a computer -- Instructions

Instructions specify commands to:

- Transfer information **within** a **computer** (e.g., from memory to ALU)
- Transfer of information between the **computer** and **I/O** devices (e.g., from keyboard to computer, or computer to printer)
- Perform arithmetic and logic operations (e.g., Add two numbers, Perform a logical AND).

A sequence of instructions to perform a task is called a **program**, which is stored in the memory.

**Processor fetches instructions that make up a program from the memory and performs the operations stated in those instructions.**

**What do the instructions operate upon?**

Which is NOT a characteristic of secondary memory?

- a) Non-volatile storage
- b) Cheaper per bit than primary memory
- c) Directly accessible by the CPU
- d) Provides large storage capacity

Which of the following best describes the function of the Control Unit (CU)?

- a) Performs logical comparisons of data.
- b) Generates timing and control signals to coordinate computer operations.
- c) Stores frequently accessed data and instructions.
- d) Acts as a permanent repository for system programs.

Which is NOT a characteristic of secondary memory?

- a) Non-volatile storage
- b) Cheaper per bit than primary memory
- c) Directly accessible by the CPU
- d) Provides large storage capacity

Ans: c) Directly accessible by the CPU

Which of the following best describes the function of the Control Unit (CU)?

- a) Performs logical comparisons of data.
- b) Generates timing and control signals to coordinate computer operations.
- c) Stores frequently accessed data and instructions.
- d) Acts as a permanent repository for system programs.

**Answer:** b) Generates timing and control signals to coordinate computer operations

Which of the following statements is correct regarding **primary vs secondary memory**?

- a) Primary memory is slower but larger than secondary memory.
- b) Secondary memory is volatile, whereas primary memory is non-volatile.
- c) Primary memory is directly accessible by the CPU, while secondary memory is not.
- d) Both memories are equally fast but differ in size.

Which of the following statements is correct regarding **primary vs secondary memory**?

- a) Primary memory is slower but larger than secondary memory.
- b) Secondary memory is volatile, whereas primary memory is non-volatile.
- c) Primary memory is directly accessible by the CPU, while secondary memory is not.
- d) Both memories are equally fast but differ in size.

**Answer:** c) Primary memory is directly accessible by the CPU, while secondary memory is not



# THANK YOU

---

**Team DDCO**

Department of Computer Science





# DIGITAL DESIGN AND COMPUTER ORGANIZATION

## Basic Operational Structure of computers

T2: Chapter 1: 1.1-1.4

Department of Computer Science and Engineering

# Functional Units of a Computer

## Basic Operational Concepts T2:Ch1 1.3

### Review

- Activity in a computer is governed by **instructions**.
- To perform a task, an appropriate program consisting of a list of instructions is stored in the **memory**.
- **Individual instructions** are brought from the **memory into the processor**, which executes the specified operations.
- Data to be used as **operands** are also stored in the **memory**.

# Functional Units of a Computer

## Basic Operational Concepts

An Instruction consists of two parts: An Operation code and operand/s.



ADD MLOC, R0

# Functional Units of a Computer

## Basic Operational Concepts

---

Execution steps: To add two operands

Step 1: **Fetch** the **instruction** from main memory into the processor

Step 2: **Fetch** the **operand** at location MLOC (memory location) from main memory into the processor

Step 3: **Add** the memory **operand** (i.e. fetched contents of MLOC()) to the contents of **register R0**

Step 4: **Store** the resulting **sum in R0** itself.

The original contents of R0 is overwritten.

# Functional Units of a Computer

## Basic Operational Concepts

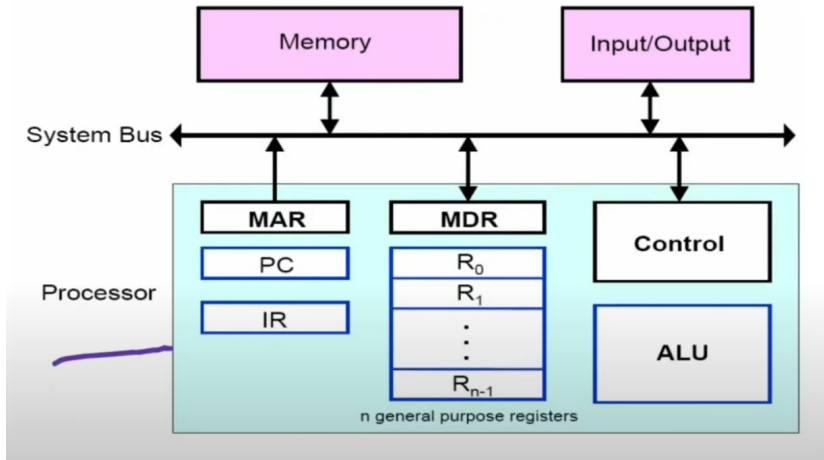
---

- Prev example combines a memory access operation with an ALU operation
- This can be realized as  
**Load LOCA,R1**  
**Add R1,R0**
- Transfers b/w the memory and the cpu are started by sending the address of the location to memory unit and issuing the appropriate control signals

# Functional Units of a Computer

## Basic Operational Concepts

### Processor and Main Memory Interaction



# Functional Units of a Computer

## Registers

---

### What is a Register?

**Definition:** A register is a **small, fast storage** location within the CPU used to temporarily hold data and instructions during processing.

### Key Characteristics:

- **Fast Access:** Registers are the fastest form of memory, as they are located directly within the CPU.
- **Limited Size:** Registers usually have a very small storage capacity, typically 32-bit or 64-bit, depending on the CPU architecture.
- **Temporary Storage:** Used to store data temporarily for fast retrieval during computations.

# Functional Units of a Computer

## Types of Registers

---

- **Instruction Register (IR)**
  - **Purpose:** Temporarily holds the instruction currently being executed by the processor.
  - **Function:** Decodes the instruction, allowing the processor to understand and execute the required operation.
- **Program Counter (PC)**
  - **Purpose:** Tracks the address of the next instruction to be executed in the program sequence.
  - **Function:** Automatically increments to point to the next instruction address after each execution.



- **General-Purpose Registers (R0 - Rn-1)**
  - **Purpose:** Temporarily store operands for quick access during instruction execution, supporting various operations like arithmetic and data transfer.
  - **Function:** Hold and manipulate data during calculations, minimizing memory access for faster CPU performance.
- **Memory Address Register (MAR)**
  - **Purpose:** Holds the memory address of the location that the CPU is about to read from or write to.
  - **Function:** Communicates directly with the memory bus for fetching or storing data.

# Functional Units of a Computer

## General Purpose Register

|                                            | 7   | 0 | Addr. |                      |
|--------------------------------------------|-----|---|-------|----------------------|
| General<br>Purpose<br>Working<br>Registers | R0  |   | 0x00  |                      |
|                                            | R1  |   | 0x01  |                      |
|                                            | R2  |   | 0x02  |                      |
|                                            | ... |   |       |                      |
|                                            | R13 |   | 0x0D  |                      |
|                                            | R14 |   | 0x0E  |                      |
|                                            | R15 |   | 0x0F  |                      |
|                                            | R16 |   | 0x10  |                      |
|                                            | R17 |   | 0x11  |                      |
|                                            | ... |   |       |                      |
|                                            | R26 |   | 0x1A  | X-register Low Byte  |
|                                            | R27 |   | 0x1B  | X-register High Byte |
|                                            | R28 |   | 0x1C  | Y-register Low Byte  |
|                                            | R29 |   | 0x1D  | Y-register High Byte |
|                                            | R30 |   | 0x1E  | Z-register Low Byte  |
|                                            | R31 |   | 0x1F  | Z-register High Byte |

# Functional Units of a Computer

## Types of Registers

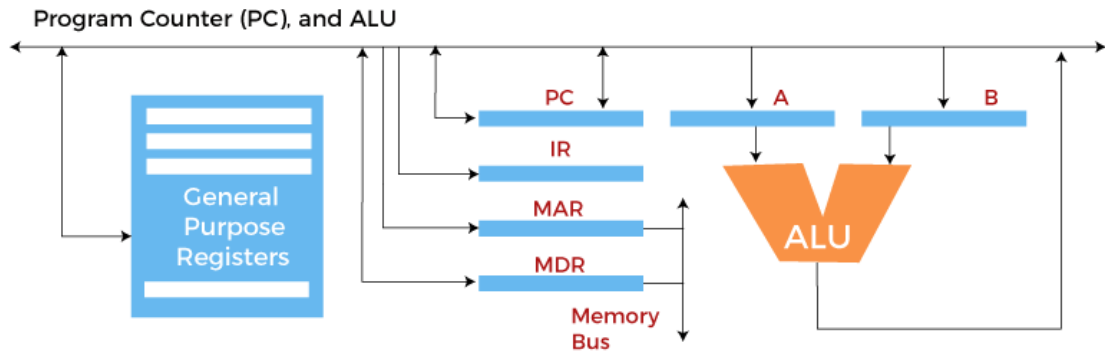
---

- **Memory Data Register (MDR)**
  - **Purpose:** Contains the actual data being transferred to or from the addressed memory location.
  - **Function:** Acts as a buffer between the CPU and memory.

# Functional Units of a Computer

## Basic Operational Concepts

### Operational Steps Associated with Processor & Main Memory Interaction



One-bus datapath

# Functional Units of a Computer

## Basic Operational Concepts

---

### How is an instruction executed ?

A fetch-decode cycle is performed for each instruction in a program.

This means that every time the CPU needs to execute a new instruction, it goes through this cycle:

1. **Fetch:** The CPU retrieves the next instruction from memory using the program counter (PC).
2. **Decode:** The CPU interprets the instruction to determine what operation it needs to perform and what data it needs to use.
3. **Execute:** The CPU carries out the instruction, which might involve performing calculations, manipulating data, or controlling other parts of the computer.

# Functional Units of a Computer

## Basic Operational Concepts

---

### How is the next instruction executed ?

After the execution of an instruction, the PC is typically incremented to point to the next instruction in the program sequence, and the cycle repeats. This continuous process allows the CPU to execute a program's instructions one by one until the program is finished.

Let us now look at this process in detail.

# Functional Units of a Computer

## Basic Operational Concepts

---

### **1. Programs reside in memory through input devices:**

Programs are loaded into the computer's memory from input devices like keyboards or storage devices (e.g., hard drives, SSDs). This step typically happens before the program starts executing.

### **2. PC is set to point to the first instruction:**

The Program Counter (PC) holds the address of the next instruction to be executed. At the start, it is initialized to point to the first instruction in the program.

### **3. The contents of PC are transferred to MAR:**

The Memory Address Register (MAR) holds the address of the memory location that is being accessed. The address from the PC is copied to MAR.

# Functional Units of a Computer

## Basic Operational Concepts

---

### 4. **A Read signal is sent to the memory:**

A signal is sent to the memory to indicate that data should be read from the address specified by the MAR.

### 5. **The first instruction is read out and loaded into MDR:**

The instruction located at the address in MAR is read from memory and placed into the Memory Data Register (MDR).

### 6. **The contents of MDR are transferred to IR:**

The Instruction Register (IR) holds the current instruction to be executed. The data in MDR (which is the fetched instruction) is transferred to IR.



# Functional Units of a Computer

## Basic Operational Concepts

---

### 7. **The instruction is ready to be decoded and executed:**

The instruction in IR is decoded to understand what action needs to be performed.

### 8. **If the instruction involves an operation to be performed by the ALU:**

- If the instruction requires arithmetic or logical operations, the necessary operands must be obtained.
- **Get operands:**
- **General-purpose register:** If the operands are in general-purpose registers, these registers are accessed directly.

# Functional Units of a Computer

## Basic Operational Concepts

---

8.

- **Memory:** If the operands are in memory, the following steps are performed:
  - **The address of the operand is transferred to MAR.**
  - **A Read signal is sent to memory to fetch the operand.**
  - **The operand is read into MDR and then transferred to the ALU.**

### 9. Perform operation in ALU:

The Arithmetic Logic Unit (ALU) performs the operation specified by the instruction using the operands it received.

# Functional Units of a Computer

## Basic Operational Concepts

---

### 10. Store the result back:

- **To general-purpose register:** If the result needs to be stored in a register, it is written to the appropriate general-purpose register.
- **To memory:** If the result needs to be stored in memory:
  - **The address for storing the result is transferred to MAR.**
  - **The result is placed in MDR.**
  - **A Write signal is sent to memory to store the result.**

**11. During the execution of the current instruction, PC is incremented to the next instruction**

# Functional Units of a Computer

## Interrupt

---

- **What is an interrupt ?**
  - An interrupt is a request from an I/O device for service by the processor.
- **Why is an interrupt necessary?**
  - Normal execution of programs may be **interrupted** if some device requires **urgent** servicing.
  - To deal with the situation immediately, the normal execution of the current program must be interrupted.

# Functional Units of a Computer

## Procedure of interrupt operation

---

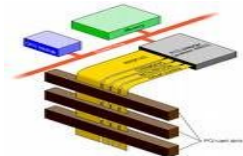
1. The I/O device raises an **interrupt signal**.
2. The processor provides the requested service by executing an appropriate **interrupt-service routine**.
3. The **state of the processor is first saved** before servicing the interrupt
4. Normally, the contents of the PC, the general registers, and some control information are stored in memory.
5. When the interrupt-service routine is completed, the **state of the processor is restored** so that the interrupted program may continue.

### What is Bus ?

- A bus is a **collection of wires** that connects different parts of a computer in an organized manner for the purpose of communicating information such as memory address, I/O address, Data, Control bits etc.

### Why is a bus required ?

- With the bus structure, speed of operation can be achieved since collection of wires in a **bus permits for transferring all bits of information at a time i.e., in parallel**. One byte or one full word at a time simultaneously on different wires.

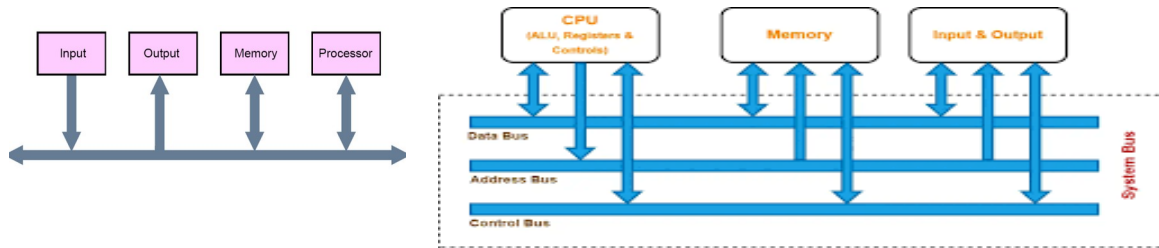


# Functional Units of a Computer

## Bus Structures

A **group** of **lines** that serves a **connecting path** for several devices is called a **bus**

- In addition to the **lines** that carry the **data**, the bus must have **lines** for **address** and **control** purposes
- The simplest way to interconnect functional units is to use a **single bus**, as shown below



# Functional Units of a Computer

## Bus Structures

---

### Single-bus

- Simplest way of interconnecting functional units.
- All units are connected to this bus.
- Can be used for only one transfer at a time, thereby only 2 units can actively use the bus at a time.
- **Low Cost and flexibility for attaching peripheral devices.**



# Functional Units of a Computer

## Bus Structures

### Drawbacks of the Single Bus Structure

- The devices connected to a bus **vary** widely in their **speed** of operation
  - Some devices are relatively **slow**, such as printer and keyboard
  - Some devices are considerably **fast**, such as optical disks
  - Memory and processor units operate are the fastest parts of a computer

### Solutions

- Efficient transfer mechanism is needed to cope with this timing problem
  - A common approach is to include **buffer registers**.
  - An another approach is to use **two-bus structure** and an additional transfer mechanism(**One bus can be used to fetch instruction other can be used to fetch data, required for execution**).

# Functional Units of a Computer

## Buffer Register

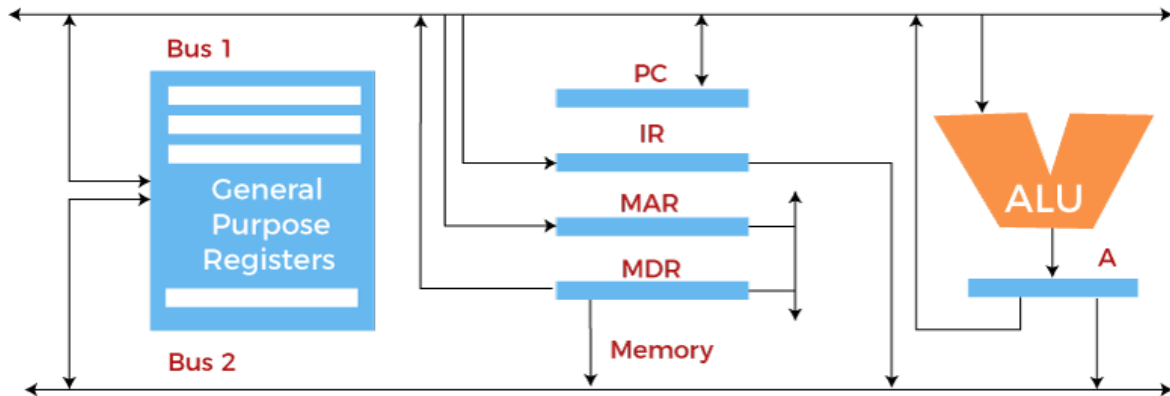
---

### Working :

- The approach is to include **buffer registers with the devices to hold the information during transfers** which smoothes out the timing difference between the cpu, memory and i/o devices
- A buffer register holds data temporarily. Buffer register compensate the timing differences among processor, memory and slow speed peripherals like Printer, Key-board, magnetic tape etc., during the transfers over a common communication path i.e., single bus.

# Functional Units of a Computer

## Two Bus Architecture



Two Bus Datapath

# Functional Units of a Computer

## Two Bus Architecture

---

**Two-Bus Structure** and an additional transfer mechanism (**One bus can be used to fetch instruction other can be used to fetch data, required for execution**).

### Working :

- The approach is to include buffer registers with the devices to hold the information during transfers which smoothes out the timing difference between the cpu, memory and i/o devices
- A buffer register holds data temporarily. Buffer register compensate the timing differences among processor, memory and slow speed peripherals like Printer, Key-board, magnetic tape etc., during the transfers over a common communication path i.e., single bus.

Which of the following registers acts as a buffer between CPU and main memory?

- a) Program Counter (PC)
- b) Memory Address Register (MAR)
- c) Memory Data Register (MDR)
- d) Instruction Register (IR)

# Functional Units of a Computer

## MCQ

Which of the following registers acts as a buffer between CPU and main memory?

- a) Program Counter (PC)
- b) Memory Address Register (MAR)
- c) Memory Data Register (MDR)
- d) Instruction Register (IR)

Answer: c) Memory Data Register (MDR)

Explanation: MDR holds the actual data being transferred to/from memory, acting as a buffer.

The Program Counter (PC) of a processor contains 0x200. The instruction at 0x200 is a branch instruction to address 0x400. Assuming the branch is taken, what will be the content of the PC after the instruction is executed?

- a) 0x200
- b) 0x201
- c) 0x400
- d) 0x401

The Program Counter (PC) of a processor contains 0x200. The instruction at 0x200 is a branch instruction to address 0x400. Assuming the branch is taken, what will be the content of the PC after the instruction is executed?

- a) 0x200
- b) 0x201
- c) 0x400
- d) 0x401

**Answer:** c) 0x400

Explanation: PC holds the address of the next instruction. For a branch instruction, if the branch is taken, the PC is loaded with the branch target (0x400).



6. An **interrupt** in a computer system is best described as:
- a) A request from an I/O device for immediate processor service
  - b) A program error caused by division by zero
  - c) A trap used to handle illegal instructions
  - d) A buffer overflow condition

An **interrupt** in a computer system is best described as:

- a) A request from an I/O device for immediate processor service
- b) A program error caused by division by zero
- c) A trap used to handle illegal instructions
- d) A buffer overflow condition

**Answer:** a) A request from an I/O device for immediate processor service

Explanation: Interrupts are signals raised by I/O devices to get urgent CPU service.

Which of the following is a major drawback of the **single-bus structure** in computer organization?

- a) High cost of connecting devices
- b) Only one transfer at a time is possible
- c) Difficult to attach peripheral devices
- d) Requires multiple buffer registers

Which of the following is a major drawback of the **single-bus structure** in computer organization?

- a) High cost of connecting devices
- b) Only one transfer at a time is possible
- c) Difficult to attach peripheral devices
- d) Requires multiple buffer registers

**Answer:** b) Only one transfer at a time is possible

Explanation: A single bus allows only two devices to communicate at a time, creating bottlenecks.



# THANK YOU

---

**Team DDCO**

Department of Computer Science



# DIGITAL DESIGN AND COMPUTER ORGANIZATION

## Number, Arithmetic Operations, And Characters Memory Locations And Address

T2: Chapter 2: 2.1-2.2

Department of Computer Science and Engineering

# Number, Arithmetic Operations, And Characters

## Outline

---

1. Number Representation
2. Addition of Positive Numbers
3. Addition And Subtractions of Signed Numbers
4. Overflow In Integer Arithmetic
5. Characters

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 1. Number Representation T2:Ch2 2.1

Department of Computer Science and Engineering



# Number, Arithmetic Operations, And Characters

## Introduction(T2- section 2.1)

---

- Programming languages are classified into three categories: Machine Level Language (MLL), Assembly Level Language (ALL), and High Level Language (HLL)
- ALL- readable representation of MLL
- Computers are built using logic circuits that operate on information represented by two- valued electrical signals . We label the two values as 0 and 1
- we define the amount of information represented by such a signal as a bit of information, where **bit** stands for **binary digit**.
- The most natural way to represent a number in a computer system is by a string of bits, called a **binary number**.
- **A text character can also be represented by a string of bits called a character code.**

# Number Representation-signed and unsigned

## Signed Integer

**Consider an n-bit vector**

$$B = b_{n-1} b_{n-2} \dots b_1 b_0$$

where  $b_i = 0$  or  $1$  for  $0 \leq i \leq n - 1$ .

This vector can represent unsigned integer values  $V$  in the range  $0$  to  $2^n - 1$ , where

$$V(B) = b_{n-1} \times 2^{n-1} + \dots + b_1 \times 2^1 + b_0 \times 2^0$$

# Number Representation-signed and unsigned

## Signed Integer

---

- 3 major representations:

**Sign and magnitude**

**One's complement**

**Two's complement**

- In all three systems, the **leftmost bit is 0 for positive and 1 for negative**
- **Positive numbers have identical representations in all systems**, but negative values have different representations

Example:

4-bit machine word

16 different values can be represented

Roughly half are positive, half are negative

# Number Representation

## Binary Signed Integer Representation

| $B$               | Values represented |                |                |
|-------------------|--------------------|----------------|----------------|
| $b_3 b_2 b_1 b_0$ | Sign and magnitude | 1's complement | 2's complement |
| 0 1 1 1           | + 7                | + 7            | + 7            |
| 0 1 1 0           | + 6                | + 6            | + 6            |
| 0 1 0 1           | + 5                | + 5            | + 5            |
| 0 1 0 0           | + 4                | + 4            | + 4            |
| 0 0 1 1           | + 3                | + 3            | + 3            |
| 0 0 1 0           | + 2                | + 2            | + 2            |
| 0 0 0 1           | + 1                | + 1            | + 1            |
| 0 0 0 0           | + 0                | + 0            | + 0            |
| 1 0 0 0           | - 0                | - 7            | - 8            |
| 1 0 0 1           | - 1                | - 6            | - 7            |
| 1 0 1 0           | - 2                | - 5            | - 6            |
| 1 0 1 1           | - 3                | - 4            | - 5            |
| 1 1 0 0           | - 4                | - 3            | - 4            |
| 1 1 0 1           | - 5                | - 2            | - 3            |
| 1 1 1 0           | - 6                | - 1            | - 2            |
| 1 1 1 1           | - 7                | - 0            | - 1            |

# Number Representation

## Summary of the table

- **SIGN & MAGNITUDE SYSTEM:** Negative value is obtained by changing the sign bit (MSB)

Range:  $-(2^{n-1}) - 1$  to  $+2^{n-1} - 1$

- **SIGNED 1'S COMPLEMENT:** Negative number is obtained by complementing each bit of the corresponding positive number i.e  $(2^n - 1) - N$

Range:  $-(2^{n-1}) - 1$  to  $+2^{n-1} - 1$

- **SIGNED 2'S COMPLEMENT:** Negative number is obtained by taking 2's complement of positive number i.e  $2^n - N$
- Range:  $-(2^{n-1})$  to  $+(2^{n-1} - 1)$

# Addition of Positive Number

T2:Ch2 2.1

|       |     |       |       |       |  |       |
|-------|-----|-------|-------|-------|--|-------|
| 0     |     | 1     |       | 0     |  | 1     |
| + 0   | + 0 | + 1   | + 1   |       |  |       |
| ----- |     | ----- | ----- | ----- |  | ----- |
| 0     |     | 1     | 1     |       |  | 1 0   |
| ----- |     | ----- | ----- | ----- |  | ----- |

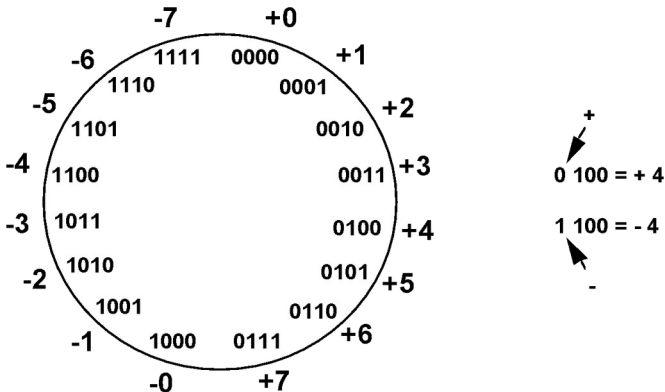
Addition of 1-bit numbers

Carry-out

# Addition And Subtraction of Signed Numbers

## Sign and Magnitude Representation

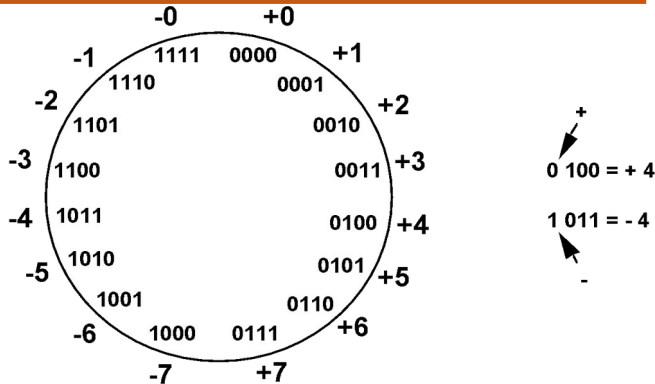
T2:Ch2 2.1



High order bit is sign: 0 = positive (or zero), 1 = negative  
 Three low order bits is the magnitude: 0 (000) thru 7 (111)  
 Range =  $-(2^{n-1})-1$  to  $+2^{n-1} -1$   
 Two representations for 0

# Addition And Subtraction of Signed Numbers

## One's Complement Representation

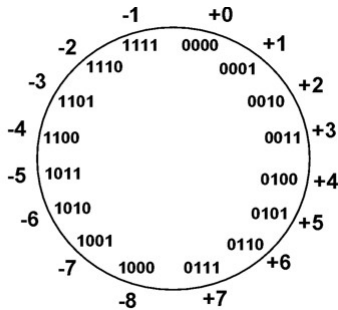


- In 1's complement -ve value is obtained by complementing each bit of the corresponding +ve value
- The operation of forming 1's complementing is equivalent to subtracting that no. from  $2^n - 1$
- Still two representations of 0! This causes some problems



# Addition And Subtraction of Signed Numbers

## Two's Complement Representation



*Modulo 16 for 2's complement representation*

- The operation of forming 2's complementing is equivalent to subtracting that no. from  $2^n$
- 2's complement of a no. can be obtained by adding 1 to the 1's complement of that no.
- Only one representation for 0
- One more negative number than positive number

# Addition And Subtraction of Signed Numbers

## Importat Points

---

### Rules for Addition and Subtraction of $n$ -bit Signed Numbers (2's Complement System)

#### 1.Addition:

To add two numbers, add their  $n$ -bit representations, ignoring the carry-out signal from the **most significant bit (MSB)** position. The sum will be the algebraically correct value in the 2's-complement representation as long as the answer is in the range:  $-2^{n-1}$  *through*  $+2^{n-1} - 1$

#### 2.Subtraction:

To subtract two numbers  $X$  and  $Y$  (i.e., to perform  $X - Y$ ), form the 2's-complement of  $Y$  and then add it to  $X$ , as in rule 1. Again, the result will be the algebraically correct value in the 2's-complement representation system if the answer is in the range:

$$-2^{n-1} \text{ through } +2^{n-1} - 1$$

# Addition And Subtraction of Signed Numbers

---

- For signed arithmetic operations 2's complement format is used
- For signed arithmetic operation sign and magnitude and 1's complement format is not suitable
- To represent signed number in 2's complement using larger number of bits repeat the sign as many times needed which is called sign extension

# Addition And Subtraction of Signed Numbers

## Addition using 2's Complement

---

$$\begin{array}{r} 0111 \\ + 1101 \\ \hline \end{array}$$
$$\begin{array}{r} 10100 \\ \hline \end{array}$$

The carry out should be ignored

# Example of addition of n – bit signed numbers using 2's complement representation

+2      0010  
+3      + 0011

----  
-----

+5      0101  
----  
-----

-4      1100  
-3      + 1101

---  
-----

-7      ①1001  
---  
-----

+4      0100  
-6      +1010

----  
-----

-2      1110  
----  
-----

-6      1010  
+3      + 0011

---  
-----

-3      1101  
---  
-----

# Example of subtraction of n - bit signed numbers using 2's complement representation

$$\begin{array}{r} +4 \\ - +3 \\ \hline \\ \hline +1 \\ \hline \end{array}$$

$$\begin{array}{r} 0100 \\ - 0011 \rightarrow \\ \hline \end{array}$$

$$\begin{array}{r} 0100 \\ + 1101 \\ \hline \\ \hline 0001 \\ \hline \end{array}$$

①

$$\begin{array}{r} +5 \\ - -2 \\ \hline \\ \hline +7 \\ \hline \end{array}$$

$$\begin{array}{r} 0101 \\ - 1110 \rightarrow \\ \hline \end{array}$$

$$+ 0010$$

$$\begin{array}{r} 0101 \\ \hline \\ \hline 0111 \\ \hline \end{array}$$

# Example of subtraction of n - bit signed numbers using 2's complement representation

|       |        |   |   |       |
|-------|--------|---|---|-------|
| -3    | 1101   |   |   | 1101  |
| - -6  | - 1010 | → | + | 0110  |
| ----- | -----  |   |   | ----- |
| +3    |        |   |   | 0011  |
| ----- |        |   |   | ----- |

|       |        |   |   |       |
|-------|--------|---|---|-------|
| -4    | 1100   |   |   | 1100  |
| - +2  | - 0010 | → | + | 1110  |
| ----- | -----  |   |   | ----- |
| -6    |        |   |   | 1010  |
| ----- |        |   |   | ----- |

# Example of addition and subtraction of n - bit signed numbers using 2's complement representation leading to overflow

$$\begin{array}{r} +3 \\ - -6 \\ \hline \\ +9 \\ \hline \end{array}$$

$$\begin{array}{r} 0011 \\ - 1010 \\ \hline \end{array} \rightarrow$$

$$\begin{array}{r} 0011 \\ + 0110 \\ \hline 1001 \rightarrow -7 \\ \hline \end{array}$$

$$\begin{array}{r} -4 \\ + -5 \\ \hline \\ -9 \\ \hline \end{array}$$

$$\begin{array}{r} 1100 \\ + 1011 \\ \hline 1\ 0111 \rightarrow +7 \\ \hline \end{array}$$



## Summary

---

- It might seem that the 1's-complement representation would be just as good as the 2's-complement system. However, although complementation is easy, the result obtained after an addition operation is not always correct. The carry-out,  $c_n$ , cannot be ignored.
- If  $c_n = 0$ , the result obtained is correct.
- If  $c_n = 1$ , then a 1 must be added to the result to make it correct.
- The need for this correction cycle, which is conditional on the carry-out from the add operation, means that addition and subtraction cannot be implemented as conveniently in the 1's-complement system as in the 2's-complement system.

- When result of arithmetic operation is outside representable range, arithmetic overflow has occurred.
- Overflow will not occur when two numbers having opposite signs added
- **Overflow can occur only if two numbers having same signs are added**
- The **carry out signal** from the sign bit position **is not a sufficient indicator** of overflow when adding signed numbers
- A simple way to detect the overflow is to examine the signs of the two numbers X and Y and the sign of the result. **When both the operands X and Y have the same sign, an over flow occurs when the sign of S is not the same as the signs of X and Y**

- In addition to numbers, computers must be able to handle nonnumeric text information consisting of characters.
- Characters can be letters of the alphabet, decimal digits, punctuation marks, and so on.
- They are represented by codes that are usually eight bits long. One of the most widely used such codes is the American Standards Committee on Information Interchange (ASCII) code

**Table E.2** The 7-bit ASCII code

| Bit positions | Bit positions 654 |     |       |     |     |     |     |     |
|---------------|-------------------|-----|-------|-----|-----|-----|-----|-----|
| 3210          | 000               | 001 | 010   | 011 | 100 | 101 | 110 | 111 |
| 0000          | NUL               | DLE | SPACE | 0   | @   | P   | '   | p   |
| 0001          | SOH               | DC1 | !     | 1   | A   | Q   | a   | q   |
| 0010          | STX               | DC2 | "     | 2   | B   | R   | b   | r   |
| 0011          | ETX               | DC3 | #     | 3   | C   | S   | c   | s   |
| 0100          | EOT               | DC4 | \$    | 4   | D   | T   | d   | t   |
| 0101          | ENQ               | NAK | %     | 5   | E   | U   | e   | u   |
| 0110          | ACK               | SYN | &     | 6   | F   | V   | f   | v   |
| 0111          | BEL               | ETB | '     | 7   | G   | W   | g   | w   |
| 1000          | BS                | CAN | (     | 8   | H   | X   | h   | x   |
| 1001          | HT                | EM  | )     | 9   | I   | Y   | i   | y   |
| 1010          | LF                | SUB | *     | :   | J   | Z   | j   | z   |
| 1011          | VT                | ESC | +     | ;   | K   | [   | k   | {   |
| 1100          | FF                | FS  | ,     | <   | L   | /   | l   |     |
| 1101          | CR                | GS  | -     | =   | M   | ]   | m   | }   |
| 1110          | SO                | RS  | .     | >   | N   | ^   | n   | ~   |
| 1111          | SI                | US  | /     | ?   | O   | _   | o   | DEL |

In sign-and-magnitude representation using 4 bits, the result of  $(+5) + (-5)$  is:

- a) 0000 (unique zero)
- b) 1000 (alternate zero)
- c) Either 0000 or 1000 (two representations of zero)
- d) 1111

In sign-and-magnitude representation using 4 bits, the result of  $(+5) + (-5)$  is:

- a) 0000 (unique zero)
- b) 1000 (alternate zero)
- c) Either 0000 or 1000 (two representations of zero)
- d) 1111

**Answer:** c) Either 0000 or 1000 (two representations of zero)

Which of the following statements about overflow in signed integer arithmetic is correct?

- a) Overflow can occur when adding two numbers of opposite signs.
- b) Overflow occurs when carry-out from MSB is 1.
- c) Overflow occurs when adding two numbers of same sign and the result has opposite sign.
- d) Overflow occurs when sign extension is applied.

Which of the following statements about overflow in signed integer arithmetic is correct?

- a) Overflow can occur when adding two numbers of opposite signs.
- b) Overflow occurs when carry-out from MSB is 1.
- c) Overflow occurs when adding two numbers of same sign and the result has opposite sign.
- d) Overflow occurs when sign extension is applied.

**Answer:** c) Overflow occurs when adding two numbers of same sign and the result has opposite sign.

If numbers  $X = 0111$  and  $Y = 1101$  are added in 2's complement 4-bit representation, what is the result?

- a) 0100 with overflow
- b) 0010 with no overflow
- c) 1110 with overflow
- d) 1000 with no overflow



If numbers  $X = 0111$  and  $Y = 1101$  are added in 2's complement 4-bit representation, what is the result?

- a) 0100 with overflow
- b) 0010 with no overflow
- c) 1110 with overflow
- d) 1000 with no overflow

**Answer:** a) 0100 with overflow



# THANK YOU

---

**Team DDCO**

Department of Computer Science



# DIGITAL DESIGN AND COMPUTER ORGANIZATION

## Memory Locations And Addresses

---

T2: Chapter 2: 2.2

Department of Computer Science and Engineering

# Memory Locations And Addresses

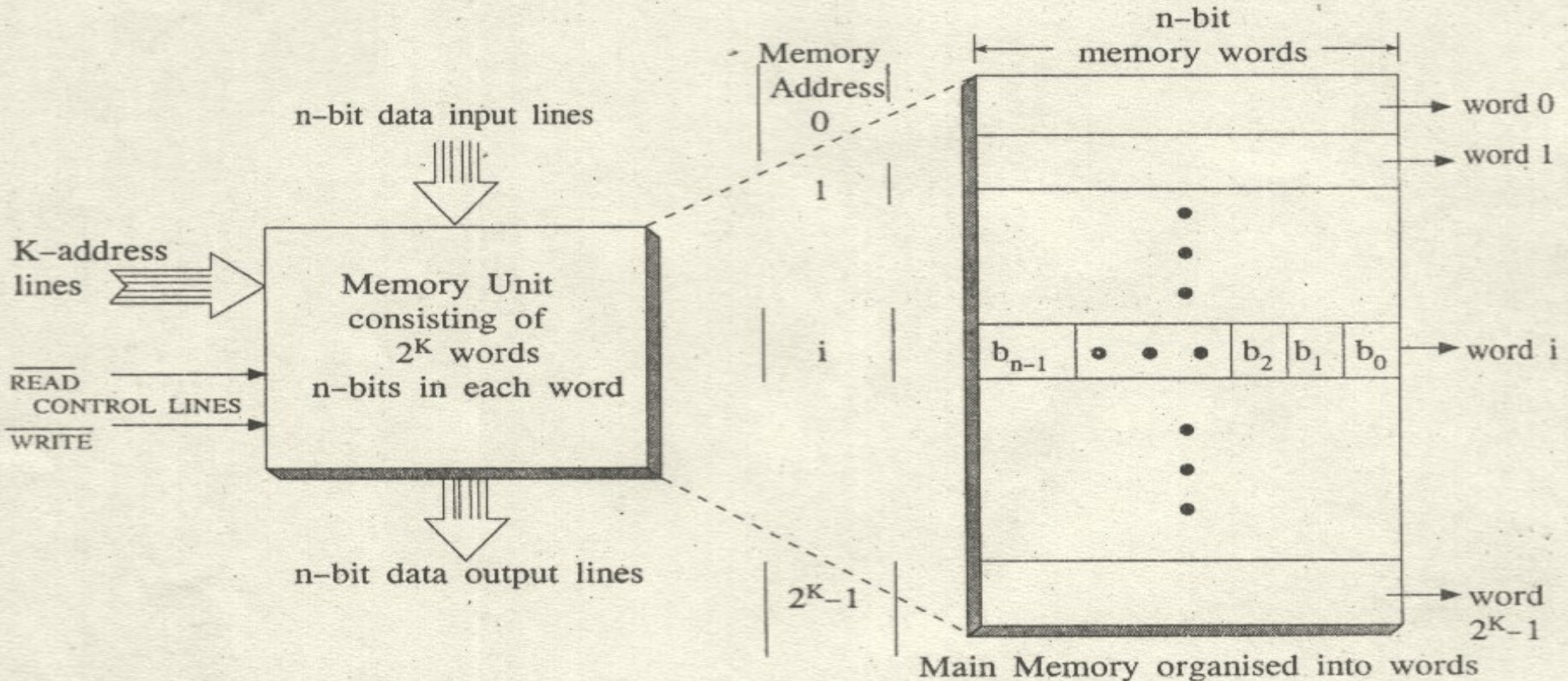
## Introduction(T2- section 2.2)

- Main memory is a collection of semiconductor storage cells,
- Each Cell can store one bit of information say a “1” or a “0” bit.
- Memory stores binary information in groups of bits called **words**.
- Memory word is an addressable location to store a number, characters as data or an instruction in binary.
- The length of a memory word can be specified as n-bits per word called the **word length**.
- Modern computers have word length of the order 16-bits to 64-bits.
- 8 bits= 1 byte

- Each word of information to be stored or retrieved from memory requires an identifying name called its **Address**.
- Special input lines called **Address lines (wires)** select each and every word in memory distinctly.
- For which each word in memory is assigned an identification number, called as address starting from 0 and continues 1,2,3,.....,up to  $2^k$  where k is the number of address lines.
- The selection of a specific memory word is achieved by applying the K-bit binary address to the address lines.
- The address range from 0 to  $2^k - 1$  constitute **address space** comprising  $2^k$  memory words.

# Memory Locations And Addresses

## Memory Organization and Addressing



# Memory Locations, Addresses, and Operation

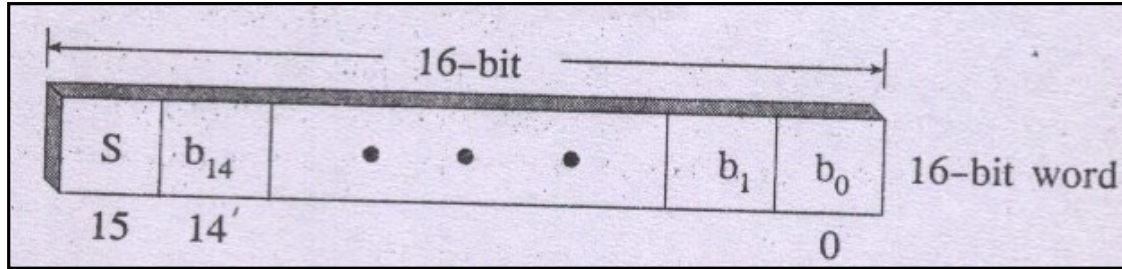
- It is impractical to assign distinct addresses to **individual bit** locations in the memory.
- The most practical assignment is to have successive addresses refer to successive byte locations in the memory - **byte-addressable memory**.
- **Byte locations have addresses 0, 1, 2, ...** If word length is 32 bits, they successive words are located at addresses 0, 4, 8,...
- **1 Tera=  $2^{40}$**
- **1Giga=  $2^{30}$**
- **1 Mega=  $2^{20}$**
- **1 Kilo=  $2^{10}$**
- **$2^{24}$  can also be called 16MB**

# Memory Locations, Addresses, and Operation

## Encoding of Information

### Numbers

- The length of the operand depends on the specific data type. 16 - bit **16 - bit memory word pattern** to hold integer numbers.



Bit 15 indicate sign bit → If it is 0, number is +ve

If it is 1, number is -ve

Binary number 0000 0010 0001 1101 is interpreted as a decimal +541



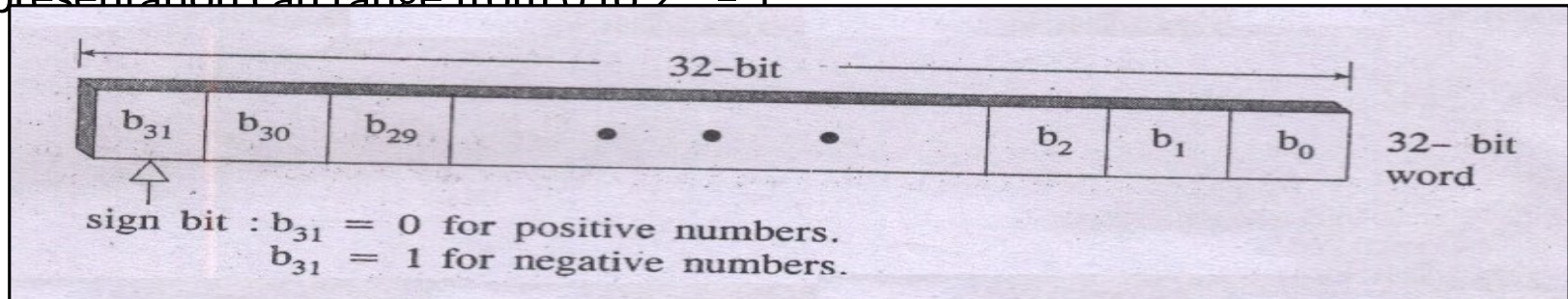
# Memory Locations, Addresses, and Operation

## Encoding of Information

- 32-bit memory word pattern to hold integer numbers
- Magnitude of the number can be computed using binary positional weight notation as:

$$= b_{30} \times 2^{30} + b_{29} \times 2^{29} + \dots + b_1 \times 2^1 + b_0 \times 2^0$$

Magnitude represented in 32-bit memory words using signed – magnitude representation can range from 0 to  $2^{31} - 1$

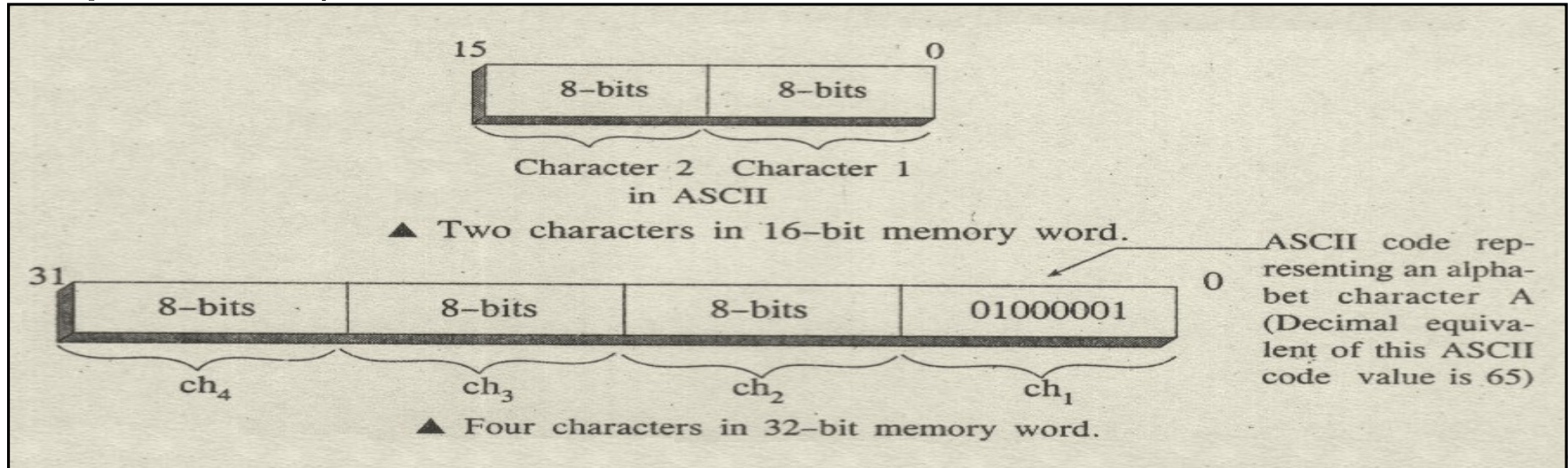


# Memory Locations, Addresses, and Operation

## Characters

### Characters

- A memory word can also store character information such as alphabets, decimal digits, symbols like +, -, \$, ?, punctuation marks like ; , ' " etc., Such characters can be represented using a 7-bit ASCII code. **Memory Word Bit pattern** to represent characters

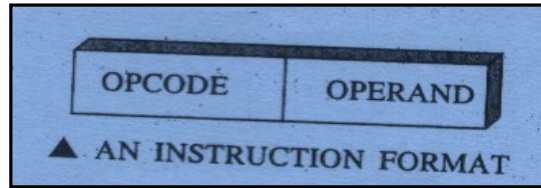


# Memory Locations, Addresses, and Operation

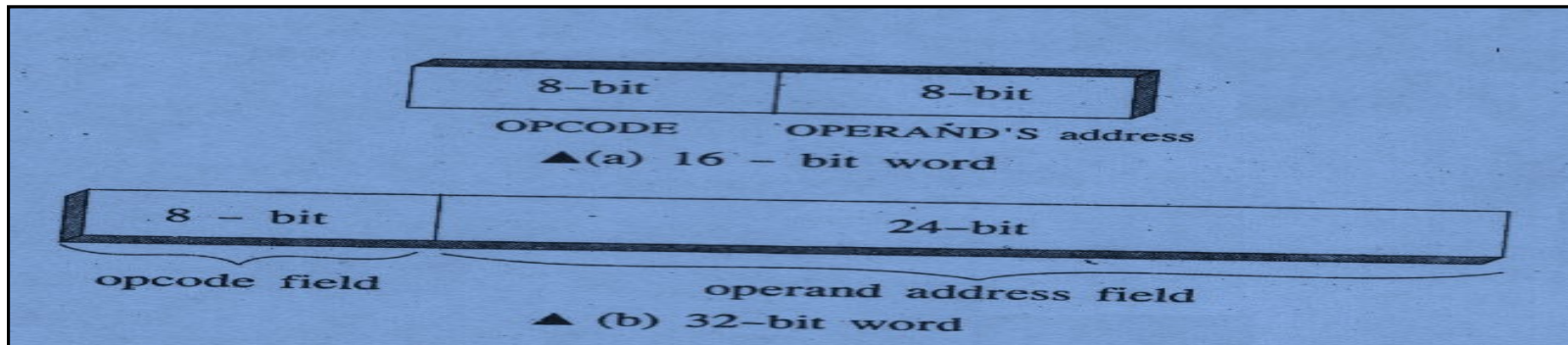
## Instructions

### Instructions

- Program instructions are also stored in main memory words. An instruction has two parts : **operation code (opcode)** and **operand field**:



- A machine instruction format in 16-bit and 32-bit memory words:



# Memory Locations, Addresses, and Operation

## Instructions

- The magnitude of the number in 16-bit memory word is computed as
- $$= b_{14} \times 2^{14} + b_{13} \times 2^{13} + \dots + b_1 \times 2^1 + b_0 \times 2^0$$

For a number in signed – magnitude representation,  
where in **bit 15( $b_{15}$ )** specify a sign bit, a 0 or 1.

- Now Binary number **0000 0010 0001 1101** has the equivalent magnitude in decimal as:

$$\begin{aligned} &= 0 \times 2^{14} + 0 \times 2^{13} + 0 \times 2^{12} + 0 \times 2^{11} + 0 \times 2^{10} + 1 \times 2^9 + 0 \times 2^8 + 0 \times 2^7 \\ &\quad + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \end{aligned}$$

$$= 512 + 16 + 8 + 4 + 1 = +541$$

# Memory Locations And Address

## Outline

---

1. Byte Addressability
2. Big-Endian And Little-Endian Assignments
3. Word Alignment
4. Accessing Numbers, Characters, And Character Strings

- If the smallest addressable unit in a computer is a word, then such computers are said to be **word addressable computer**.
- A computer in which each byte stored can be addressed individually is called as **byte addressable computer**.
- If the word length of the given computer machine is **32 -bit** words, then sequence of memory words can be located subsequently at addresses 0 (word 0), 4 (word 1), 8 (word 2)...
- Each such word contains 4 bytes.

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 2. Big-Endian & Little-Endian Assignments T2:Ch2 2.2

Department of Computer Science and Engineering

# Big-Endian And Little-Endian Assignment

Two kinds of representations in assigning bytes in a memory word are:

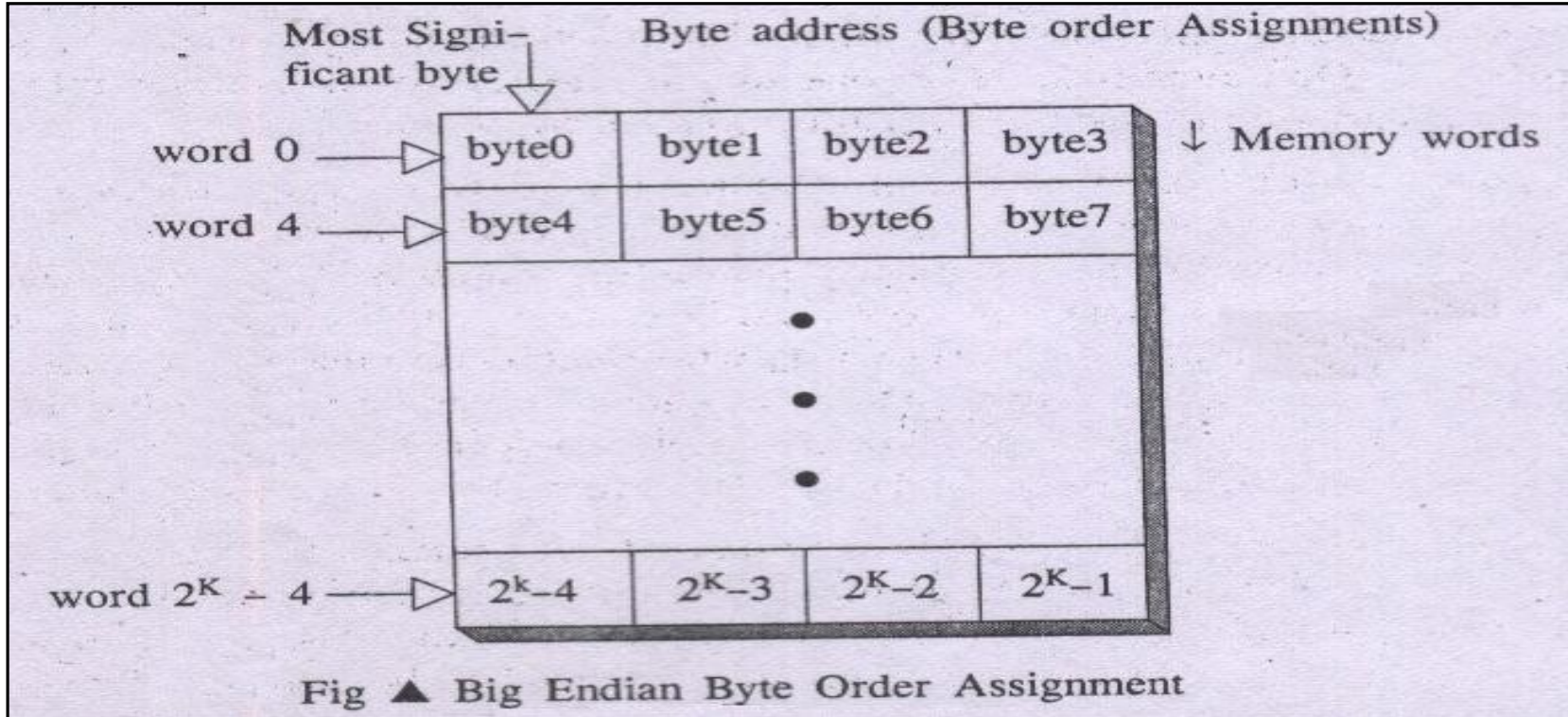
- (1) Big-endian assignment
- (2) Little-endian assignment

➤ **Big - endian assignment:** Here the bytes are assigned a number starting with most significant byte (left most byte) and successively next bytes in that word. Also the word is given the same memory address as its most significant byte.

➤ Big - Endian scheme of address assignment is used in 68000(*CISC processor*) processor and Power PC (*RISC processor*) processors.



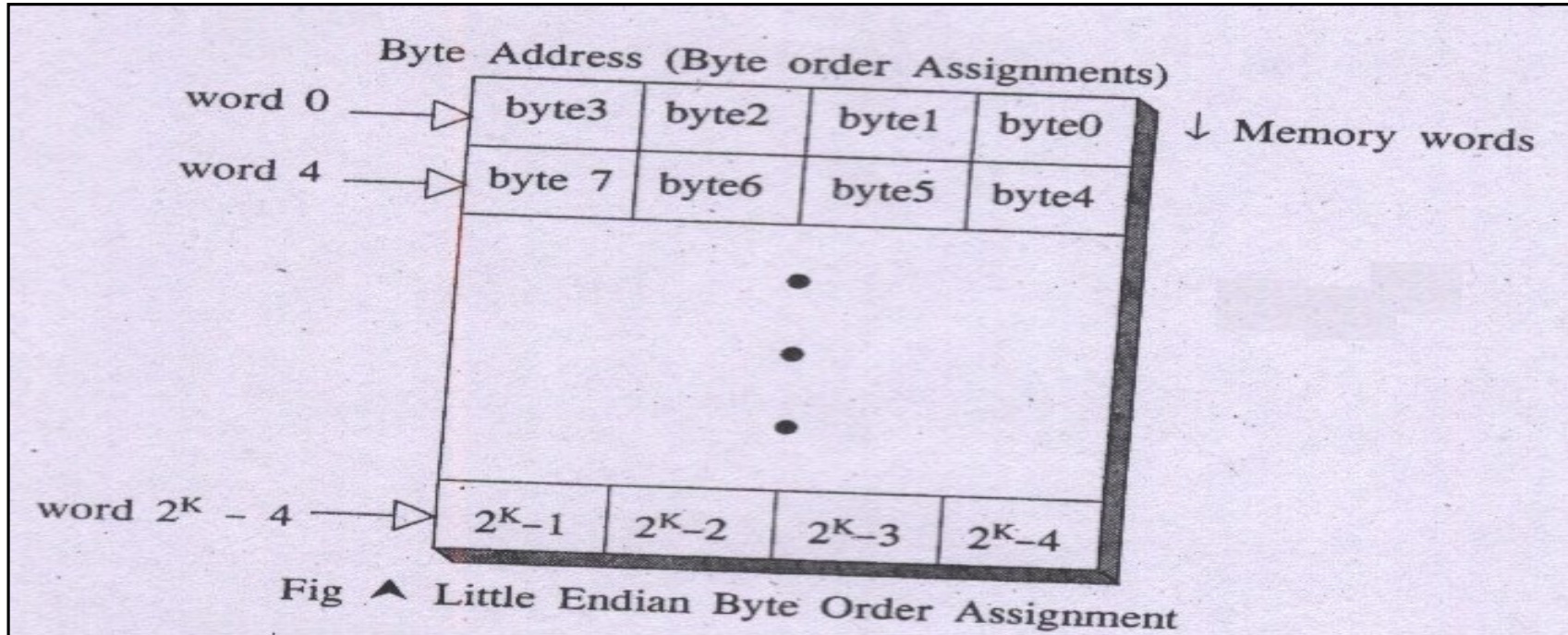
# Big-Endian Assignment



# Little-Endian Assignment

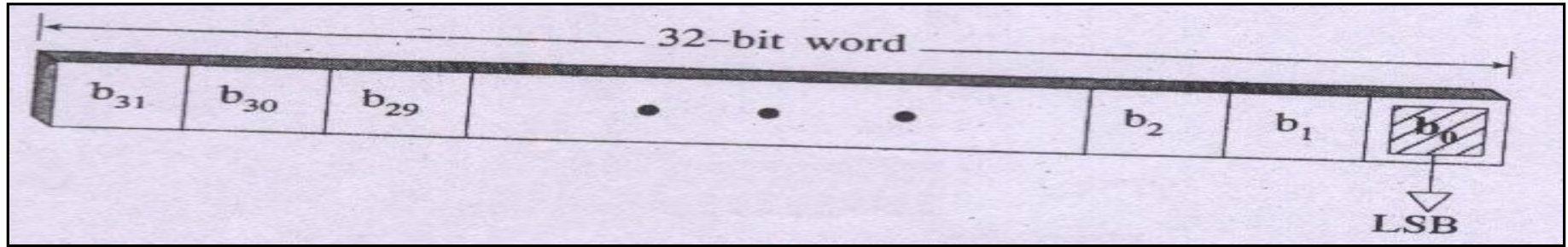
In this scheme byte 0 appears as the right most byte of word 0.

**Low order bytes represents least significant bytes or right most bytes.**

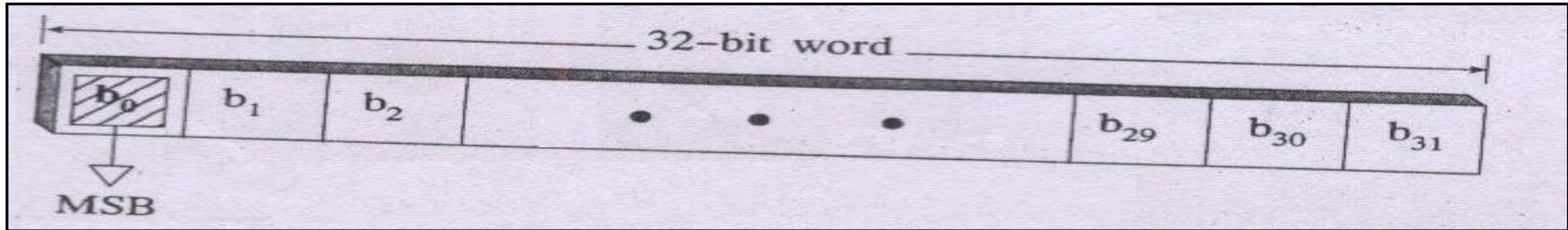


# Conventions for numbering bits in a memory word

For example 68000 processor uses a convention in which bit 0 (**b0**) is treated as least significant bit (**LSB**) of a word.

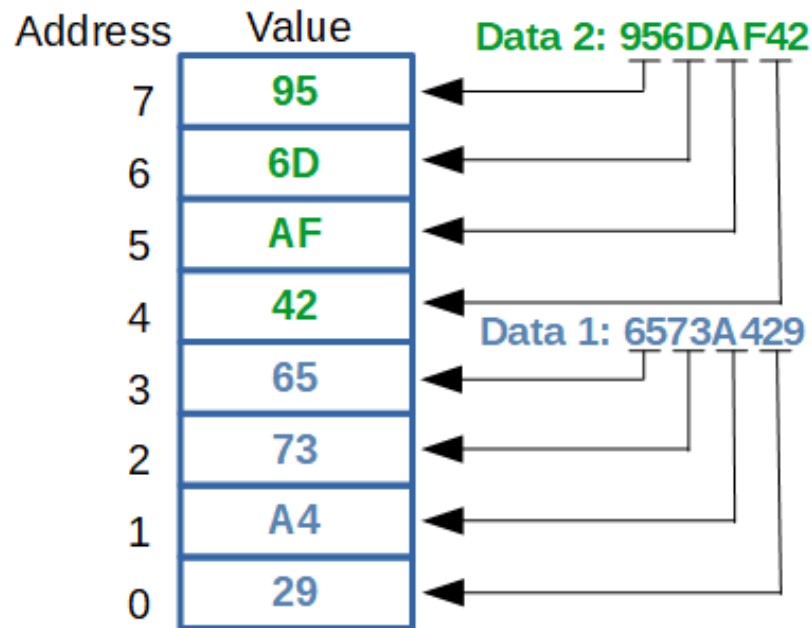


➤ Using the other convention, Power PC for example treats bit 0 (**b0**) as the most significant bit (**MSB**) of a memory word.



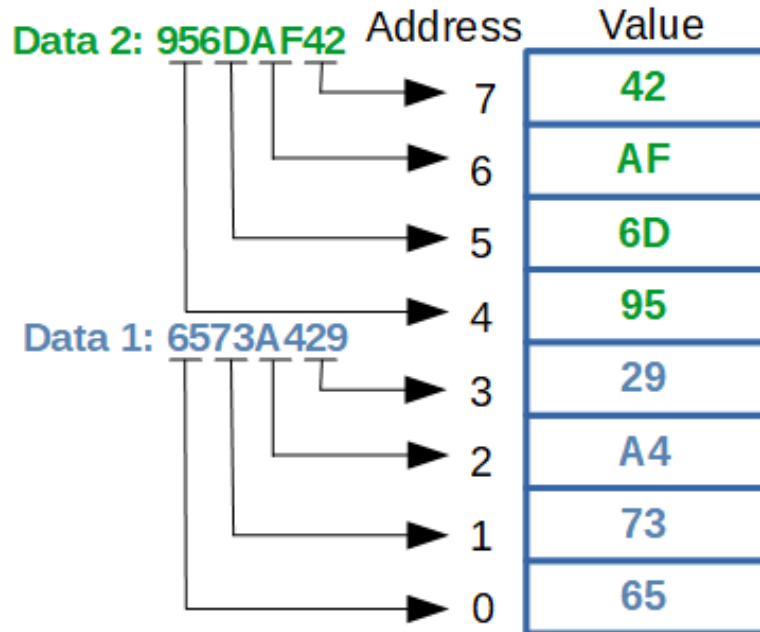
# Important

## Memory



Little-endian format

## Memory



Big-endian format

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 3. Word Alignment T2:Ch2 2.2

Department of Computer Science and Engineering



# Word Alignment

It refers to grouping of bytes in sequence in a given memory unit.

➤ For a computer with 32-bit word length

word 0 at address 0 (comprising bytes 0,1,2,3),

word 1 at address 4 (comprising bytes 4,5,6,7)

word 2 at address 8 (comprising bytes 8,9,10,11) ...

➤ Similarly for a computer with 16-bit word length:

word 0 at address 0 (comprising bytes 0,1),

word 1 at address 2 (comprising bytes 2,3)

word 2 at address 4 (comprising bytes 4,5) ...

➤ Similarly for a computer with 64-bit word length:

word 0 at address 0 (comprising bytes 0,1,2,3,4,5,6,7),

word 1 at address 8 (comprising bytes 8,9,10,11,12,13,14,15)

word 2 at address 16 (comprising bytes 16,17,18,19,20,21,22,23) ...

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

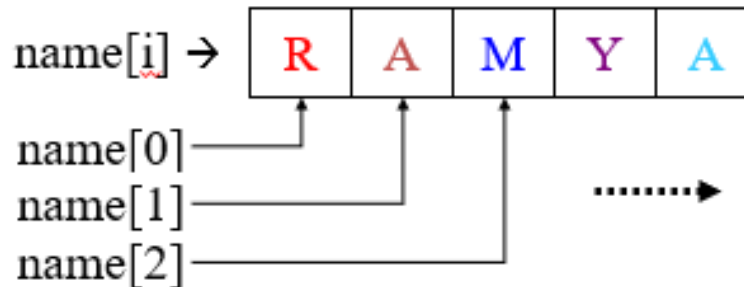
---

## 4. Numbers, Character and Character Strings T2:Ch2 2.2

Department of Computer Science and Engineering

# Numbers, Character and Character Strings

- Numbers can be accessed from memory unit by specifying its **word address** or multiple byte address in memory.
- Character constants occupy *single byte* in memory. Therefore they can be accessed specifying their **byte address**.
- For example a character string **RAMYA** in an array of char can be accessed at byte addresses in memory say bytes 0,1,2,3 & 4





1. A system has 1K words of 16-bit memory. How many address lines are required to address the memory?

- a) 8   b) 10   c) 16   d) 32

2. 12. Consider a computer with 32-bit word length using **Big-Endian** format. If a 32-bit integer is stored at address 1000, which byte holds the Most Significant Byte (MSB)?

- a) Address 1000  
b) Address 1001  
c) Address 1002  
d) Address 1003

1. A system has 1K words of 16-bit memory. How many address lines are required to address the memory?

- a) 8   b) 10   c) 16   d) 32

**Answer:** b) 10

(Reason:  $2^{10} = 1024 \rightarrow K$  words, independent of word size.)

2. 12. Consider a computer with 32-bit word length using **Big-Endian** format. If a 32-bit integer is stored at address 1000, which byte holds the Most Significant Byte (MSB)?

- a) Address 1000  
b) Address 1001  
c) Address 1002  
d) Address 1003

**Answer:** a) Address 1000

(Big-endian  $\rightarrow$  MSB stored at lowest memory address.)

A memory unit has 16 address lines. If the memory is byte-addressable, what is the maximum memory capacity?

- a) 64 KB
- b) 128 KB
- c) 256 KB
- d) 512 KB

A computer uses 32-bit word length. In a byte-addressable memory, what are the addresses of the first four words?

- a) 0, 1, 2, 3
- b) 0, 4, 8, 12
- c) 0, 2, 4, 6
- d) 0, 8, 16, 24

A memory unit has 16 address lines. If the memory is byte-addressable, what is the maximum memory capacity?

- a) 64 KB
- b) 128 KB
- c) 256 KB
- d) 512 KB

**Answer:** c) 256 KB

(Reason:  $2^{16} = 65,536$  addresses  $\rightarrow$  64K bytes = 64 KB.)

A computer uses 32-bit word length. In a byte-addressable memory, what are the addresses of the first four words?

- a) 0, 1, 2, 3
- b) 0, 4, 8, 12
- c) 0, 2, 4, 6
- d) 0, 8, 16, 24

**Answer:** b) 0, 4, 8, 12 (Reason: Each word = 4 bytes; hence addresses increase by 4 for successive words.)

# Think about it

A computer system uses a **32-bit word length** with a **byte-addressable memory**. The system supports both **Big-Endian** and **Little-Endian** formats. The system has **20 address lines**.

- Calculate the total size of the memory in bytes and words.
- If an integer 0x12345678 is stored at starting address 1000, show how the bytes will be stored in **Big-Endian** and **Little-Endian** format.
- Demonstrate how **word alignment** affects storage by showing the addresses of the first four words.
- If a character string "GATE" is stored beginning at address 2000, show how it is represented in memory.
- Briefly explain why byte-addressable memory and endianness conventions are important for **system compatibility**.



# THANK YOU

---

**Team DDCO**

Department of Computer Science



# DIGITAL DESIGN AND COMPUTER ORGANIZATION

## Addressing Modes T2: Chapter 2: 2.5

---

Department of Computer Science and Engineering

## Introduction(T2- section 2.5)

---

- Situations like **program looping**, **branching** demand convenient ways of locating memory words or memory operands.
- This convenience is being offered by different instructions (of a particular instruction set as defined by the **Microprocessor**) via various **Addressing Modes** they support.
- Suppose a given memory operand address is **too large to fit into a given instruction format** (i.e., in operand field), then it is the addressing mode (say **Register Indirect Mode** ) that can resolve this problem.



# Addressing Modes

## Introduction

---

### What is Addressing Mode . . . ?

The different ways in which the location of an operand is specified in an instruction are referred to as *addressing modes*

# Addressing Modes

## Introduction

### What is Addressing Mode . . . ?



- The addressing mode specifies *a rule or method for interpreting or modifying the address field of the instruction before the operand is actually accessed for manipulation.*
- *It is concerned with the different ways in which the location of an operand can be specified in a given instruction.*
- Various schemes for specifying *addresses of operands* in an instruction have been introduced. Such schemes are collectively known as *Addressing Modes.*

# Addressing Modes

## Introduction

---

### Advantages of Addressing modes :

- To be able to reference large number of memory location in main memory or for some system in *virtual memory*.
- To *reduce number of bits* in the address field of an instruction
- To extend programming convenience to the user / programmer by providing such facilities as *indexing of set of data values, counters for loop control, pointers to memory locations*

# Addressing Modes

## Outline

---

1. Immediate mode
2. Absolute (Direct) mode
3. Register mode
4. Indirect mode
5. Indexed mode
6. Relative mode
7. Auto increment mode
8. Auto decrement mode

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

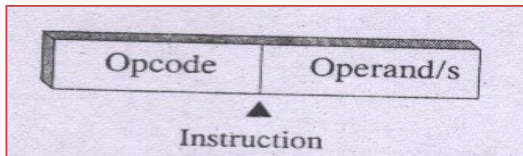
---

## 1. Immediate mode T2:Ch2 2.5

Department of Computer Science and Engineering

# Immediate mode

- Here the operand is specified in the instruction itself. An instruction that follows immediate mode has an **operand field** rather than an address field.



For example:

Move 50<sub>immediate</sub>, R0

|    |    |
|----|----|
|    | R2 |
|    | R1 |
| 50 | R0 |

- A common convention says, a symbol # has to precede the value of an immediate operand

Move #50, R0

# Immediate mode

## Example

---

Write snippet of code for  $A=B+6$

Move B,R1

Add #6,R1

Move R1,A

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 2. Absolute (Direct) mode

T2:Ch2 2.5

Department of Computer Science and Engineering



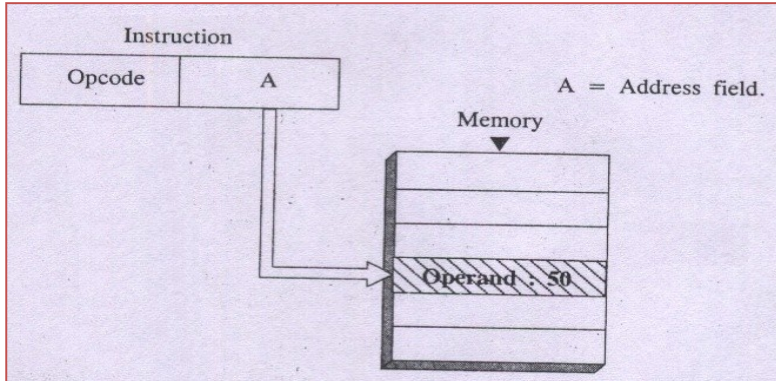
We accessed an operand by specifying the name of the register or the address of the memory location where the operand is located. The precise definitions of these two modes are:

## Addressing Modes

- **Register Mode** – The operand is the contents of a processor register; the name (address) of the register is given in the instruction.
- **Absolute Mode** – The operand is in a memory location; the address of this location is given explicitly in the instruction. (In some assembly languages, this mode is called *Direct*.)

# Absolute (Direct) mode

➤ Here **operand resides in Memory** and its address is given explicitly in the address field of an instruction. This scheme need only one memory reference in addition to instruction fetch cycle and no further calculation is required to compute operand address.



# Absolute (Direct) mode

---

- Direct addressing scheme is **simple to use** and easy to implement without the requirement for additional hardware.
- However it offers only **limited memory address space**.

## ➤ Examples

**Move** LOC,**R0**

**Load** **P**

**Store** **S**

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

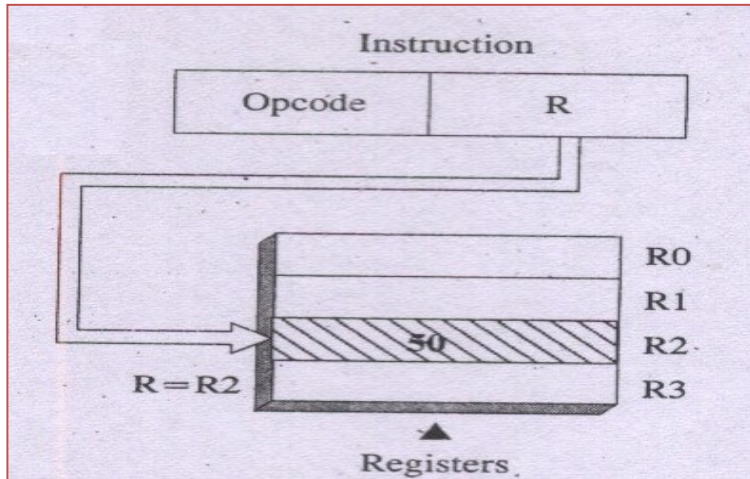
## 3. Register mode T2:Ch2 2.5

Department of Computer Science and Engineering

# Register Addressing mode

➤ In this scheme, the operand is the contents of a register appearing in the instruction i.e  $A=R$

➤ example  
Add R1,R2



# Register Addressing mode

---

## Advantages of this scheme :

- ✓ No memory reference
- ✓ A few bit address to indicate register location
- ✓ Speedy execution since register is inside the processor  
& has low access time.

## Disadvantages of this scheme :

- ✓ Limited address space as number of registers are less  
in many of the processors.

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 4. Indirect mode

T2:Ch2 2.5

Department of Computer Science and Engineering

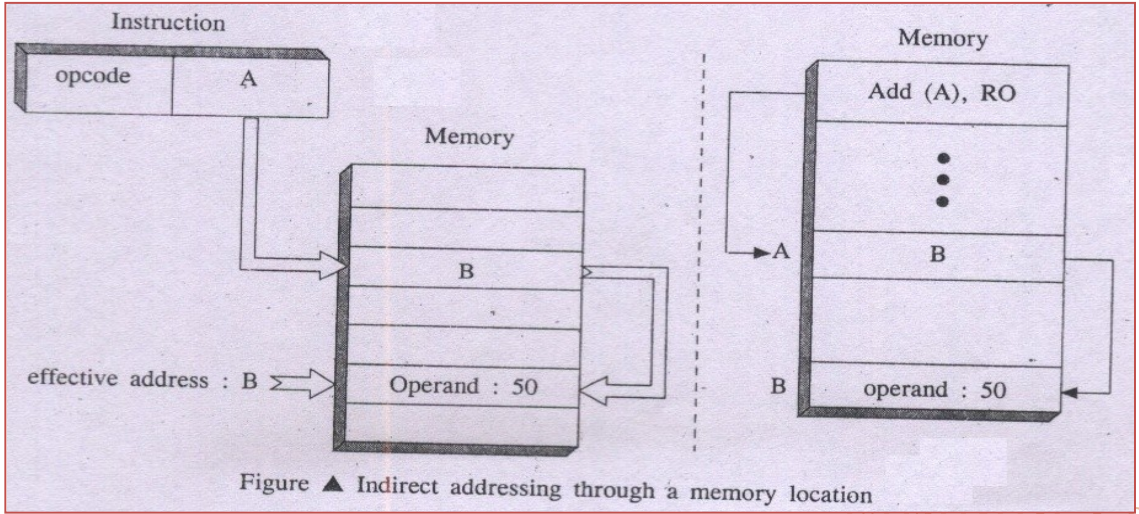
# Indirect Addressing mode

---

- The effective address of the operand is the contents of a register or memory location whose address appears in the instruction
- By referring to this address (EA), the required operand can be fetched from memory.
- Example: **Add (A), R0** i.e.  $EA = (A)$  i.e. contents of **A** is **B**  
Here **B** is effective address of desired operand in memory

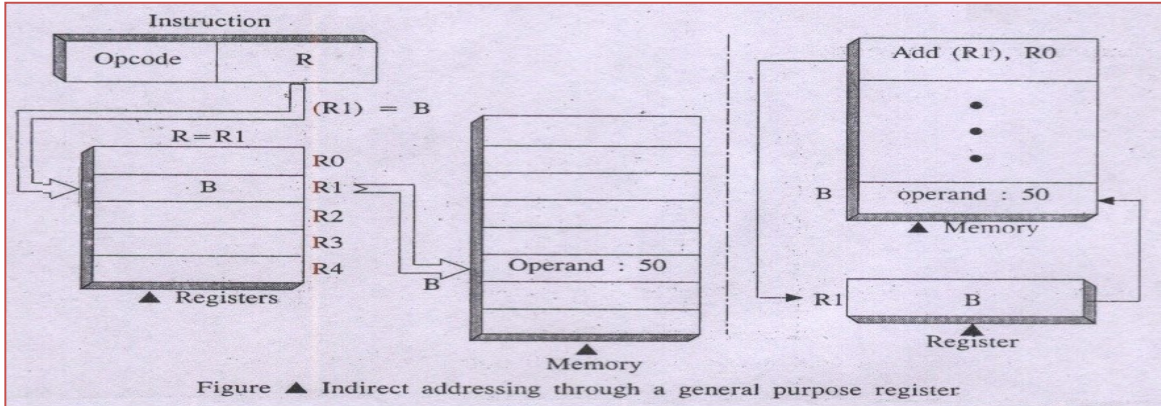


# Indirect Addressing mode



# Register Indirect Addressing mode

- Here, instruction specifies a register in the CPU whose contents give the effective address of the operand in Memory.
- For example **Add (R1), R0** i.e.  $EA = (R1)$  i.e. contents of **R1** is **B**



# Indirect Addressing mode-

## Program to add list of n numbers

---

| Address | Contents        |
|---------|-----------------|
|         | Move N, R1      |
|         | Move #NUM, R2   |
|         | Clear R0        |
| LOOP    | Add (R2), R0    |
|         | Add #4, R2      |
|         | Decrement R1    |
|         | Branch > 0 LOOP |
|         | Move R0, SUM    |

# Indirect Addressing mode

The advantage of Register Indirect Addressing:

- It uses one less memory reference (memory read operation)
- Address field of the instruction uses a fewer bits to specify a register
- Register indirect addressing can be specified with

Effective Address  $EA = (R)$  i.e.  $B$

## Advantages of Indirect addressing:

a wider address range to refer to a large number of memory locations.

## Disadvantage of Indirect addressing:

2 or more memory references (memory read operations) required to fetch the desired operand in memory.

# Indirect Addressing mode

The Register or memory location that contains address of operand is called pointer

Consider C language snippet:

`A=*B`

equivalent :

`Move B,R1`

`Move(R1),A`

Using indirect addressing:

`Move (B),A`

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 5. Indexed mode T2:Ch2 2.5

Department of Computer Science and Engineering

# Indexed Addressing mode

- Here, effective address of the operand is generated by adding a constant value to the contents of a register.
- This constant value may be either an *offset value* called *displacement* or *beginning address* of data/operand array in main memory (*Base*).
- Indexed addressing mode is symbolically represented as **X(R)**

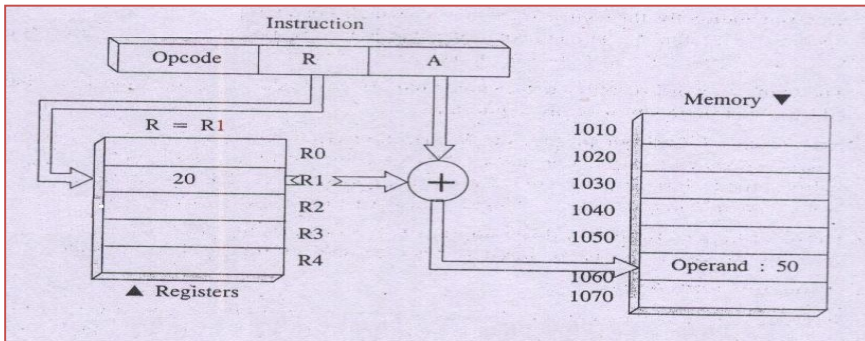
👉 Here **X** denote a constant and **R** is name of the register involved in **Indexing**.

- Effective address EA of the operand is given as

$$EA = X + [R]$$

👉 Contents of index register are not changed during the process of address generation.

# Indexed Addressing mode



A = 1040 = X; constant X corresponds to a memory location

(R) = 20 i.e. Index or offset value

EA = A + (R) = 1040 + 20

EA = 1060 at which you will find desired operand 50



# Indexed Addressing mode

You can use indexed addressing mode in two ways

- I) A constant value X defines here beginning address of operand in memory and index register Ri contains offset value (Displacement)

👉 For example 1040 in the following instruction:

Add 1040 (R1), R2

X=1040 R1=20 offset

- II) A constant value X defines an offset and index register Ri contains beginning address of operand in memory.

👉 For example 20 in the following instruction:

Add 20(R1), R2

X=20 R1=1040

# Indexed Addressing mode

- Index addressing is fast and is excellent for manipulating data structures such as **arrays** as all you need to do is set up a base address then use the index in your code to access individual elements.
- Another advantage of indexed addressing is that if the **array is re-located in memory at any point then only the base address needs to be changed**. The code making use of the index can remain exactly the same.

# Indexed Addressing mode

## Example

Example: Add 20(R1),R2

Add 2000(R3),R4

Assume that a list of scores of the student beginning at location LIST as shown in the following diagram

A 4 word memory comprises a record that stores relevant info for each student

Compute the sum of all scores obtained on each of the test and store these sums in memory locations SUM1,SUM2,SUM3

# Indexed Addressing mode

N                      5000

LIST                    5004

LIST+4                5008

LIST+8                5012

LIST+12              5016

LIST+16              5020

|                                |
|--------------------------------|
| n                              |
| Student ID                     |
| Test1                          |
| Test2                          |
| Test3                          |
| Student ID                     |
| Test1                          |
| Test2                          |
| Test3                          |
| computer organization _ unit-1 |
| .                              |
| .                              |
|                                |
|                                |

# Indexed Addressing mode

|      |           |          |            |
|------|-----------|----------|------------|
| LOOP | Move      |          | #LIST, R0  |
|      | Clear     |          | R1         |
|      | Clear     |          | R2         |
|      | Clear     |          | R3         |
|      | Move      |          | N, R4      |
|      | Add       |          | 4(R0), R1  |
|      | Add       |          | 8(R0), R2  |
|      | Add       |          | 12(R0), R3 |
|      | Add       |          | #16, R0    |
|      | Decrement | R4       |            |
|      | Branch>0  | LOOP     |            |
|      | Move      |          | R1, SUM1   |
| Move |           | R2, SUM2 |            |
| Move |           | R3, SUM3 |            |

# Indexed Addressing mode

---

## ☐ *Advantages of Index mode*

is the flexibility it offers to access relative memory locations

## ☐ *Disadvantages of Index mode*

- \* Is the complexity of computing effective address.
- \* **The instruction requires to have two address fields at least one of which is an explicit number.**

# Indexed Addressing mode

- Several variations of this basic form provide a very efficient access to memory operands in practical programming situations.
- For example, a second register may be used to contain the offset X, in which case we can write the Index mode as

**(Ri, Rj)**

The effective address is the sum of the contents of registers Ri and Rj.

- The second register is usually called the **base register**.
- This form of indexed addressing provides more flexibility in accessing operands, **because both components of the effective address can be changed.**

# Indexed Addressing mode

Another version of the Index mode uses two registers plus a constant, which can be denoted as

$$X(R_i, R_j)$$

In this case, the effective address is the sum of the constant  $X$  and the contents of registers  $R_i$  and  $R_j$ .

**This added flexibility is useful in accessing multiple components inside each item in a record, where the beginning of an item is specified by the  $(R_i, R_j)$  part of the addressing mode.**



# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 6. Relative mode T2:Ch2 2.5

Department of Computer Science and Engineering

# Relative mode

- This scheme supplies the relative position of the memory operand to be located.
- It's like index mode only but program counter register PC substitutes for base address contents
- Commonly used to specify the target address in branch instruction
- Relative Mode specify Effective Address by a notation:

$X(PC)$

- Effective address is  $EA = [PC] + X$

- **Branch > 0 loop** Here jump value / displacement  $\pm X$

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 7. Auto increment mode T2:Ch2 2.5

Department of Computer Science and Engineering

# Auto increment mode

- The EA of the operand is the contents of a register specified in the instruction
- After accessing the operand, the contents of this register are incremented **automatically to point to the next operand** in contiguous memory locations.
- Notation for Auto Increment Mode:  $(R_i) +$
- For example: **Add (R2) +, R0**
- Use of Auto Increment mode instruction eliminates the use of explicit increment instruction

# Auto increment mode

Implicitly, the increment amount is 1 when the mode is given in this form. But in a byte-addressable memory, this mode would only be useful in accessing successive bytes of some list.

To access successive words in a byte-addressable memory with a 32-bit word length, the increment must be 4. Computers that have the Autoincrement mode automatically increment the contents of the register by a value that corresponds to the size of the accessed operand.

Thus, the increment is 1 for byte-sized operands, 2 for 16-bit operands, and 4 for 32-bit operands. Since the size of the operand is usually specified as part of the operation code of an instruction, it is sufficient to indicate the Autoincrement mode as  $(Ri)+$ .

# Auto increment mode

## Program to add n numbers using autoincrement

```
    MOVE N,R1
    MOVE #NUM1,R2
    CLEAR R0
LOOP  ADD (R2)+,R0
      DECREMENT R1
      BRANCH >0 LOOP
      MOVE R0,SUM
```

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 8. Auto decrement mode T2:Ch2 2.5

Department of Computer Science and Engineering

# Auto decrement mode

- Here, content of a register specified in the instruction is
- Decrement to denote effective address of next operand in
- Successive memory location.
- Notation for Auto Decrement Mode: - (R<sub>i</sub>)
- For instance

■ Add - (R2), R0

- It allows accessing of operands in decreasing order of memory address..



# Auto decrement mode

---

The actions performed by the Autoincrement and Autodecrement addressing modes can obviously be achieved by using two instructions, one to access the operand and the other to increment or decrement the register that contains the operand address. Combining the two operations in one instruction reduces the number of instructions

# Addressing Mode

The different ways in which the location of an operand is specified in an instruction are referred to as addressing modes.

| Name                       | Assem bler syn tax          | Addressing function                           |
|----------------------------|-----------------------------|-----------------------------------------------|
| Immediate                  | #Value                      | Op erand = Value                              |
| Register                   | R <i>i</i>                  | EA = R <i>i</i>                               |
| Absolute (Direct)          | LOC                         | EA = LOC                                      |
| Indirect                   | (R <i>i</i> )               | EA = [R <i>i</i> ]                            |
|                            | (LOC)                       | EA = [LOC]                                    |
| Index                      | X(R <i>i</i> )              | EA = [R <i>i</i> ] + X                        |
| Base with index            | (R <i>i</i> , R <i>j</i> )  | EA = [R <i>i</i> ] + [R <i>j</i> ]            |
| Base with index and offset | X(R <i>i</i> , R <i>j</i> ) | EA = [R <i>i</i> ] + [R <i>j</i> ] + X        |
| Relative                   | X(PC)                       | EA = [PC] + X                                 |
| Autoincrement              | (R <i>i</i> )+              | EA = [R <i>i</i> ];<br>Increment R <i>i</i>   |
| Autodecrement              | -(R <i>i</i> )              | Decremen t R <i>i</i> ;<br>EA = [R <i>i</i> ] |



# THANK YOU

---

**Team DDCO**

Department of Computer Science



# DIGITAL DESIGN AND COMPUTER ORGANIZATION

## Memory Operations Instructions And Instruction Sequencing

T2: Chapter 2: 2.3-2.4

---

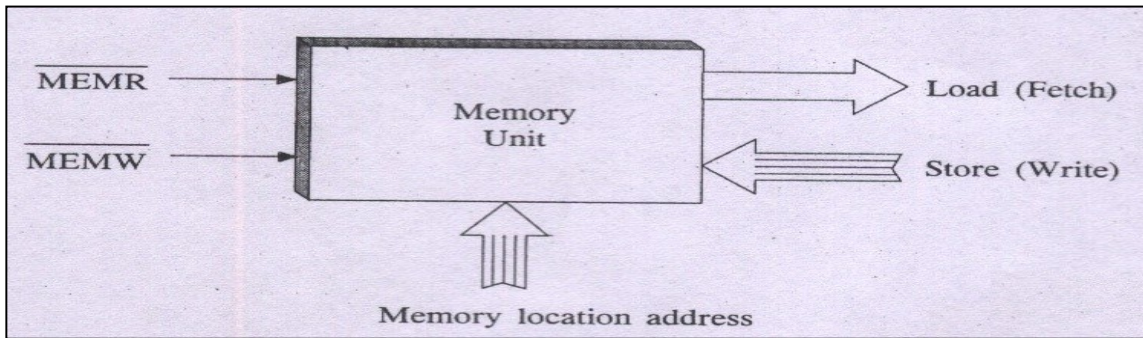
Department of Computer Science and Engineering

# Memory Operations

## Introduction(T2 -section 2.3)

While executing a program, two distinct operations associated with Processor-Memory interaction are:

- **Load** operation / Memory **Read** - (Instruction fetch / Operand fetch)
- **Store** operation / Memory **Write** - ( Write computed results)



# Memory Operations

## Load (Read Or Fetch)

---

- Transfers a copy of the contents of a specific memory location to the processor
- Processor sends address of the desired location to memory and requests that its contents be read (Issues read signal)
- Memory reads the data and sends it to the processor
- Memory contents remains unchanged

# Memory Operations

## Store Or Write

- Transfers an item of info from the CPU to a specific memory location, destroying the former contents of that location
- Processor sends the address of the desired location to the memory, together with the data to be written
- Issues write signal
- Sends the data via **data bus** and write into the selected particular memory location

# Instructions And Instruction Sequencing

## Introduction(T2 – section 2.4)

---

- An **instruction** is a command to the processor to perform a given task on data operands.
- A **program** is a set of instructions that specify **operations**, **operands** & the **sequence** by which processing has to occur.
- Thus operation of a computer is controlled by **stored program**
- In general a computer must support 4 categories of operations:
  1. **Data Movement** : To conduct I/O transfers
  2. **Data Storage** : Across Memory and processor registers.
  3. **Data Processing** : Arithmetic and logical operations
  4. **Program sequencing and control** : Test and Branch.



# Register Transfer Notation

- Data operands & Instructions are situated in main memory locations & processor registers.
- Like **mnemonics** for Opcodes, Symbolic names or label identifiers are used to name or identify such locations.
- For instances **memory location names** include **M**, **LOC**, **A**, **B**, **C**, **SUM**, **N1**, **VAR1**, **VAR2**, **NUM1**, etc.,
- Similarly **processor register names** include **R0**, **R1**, **R2**, **R3**, **R4**, **R5**, etc., and **registers** of I/O subsystem may be designated as **DATAIN**, **DATAOUT** etc.

# Register Transfer Notation

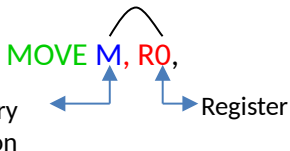
- While depicting operations, the contents of a memory location or a register are denoted by placing the corresponding name in a **pair of square brackets**.

For instance :

$R0 \leftarrow [M]$

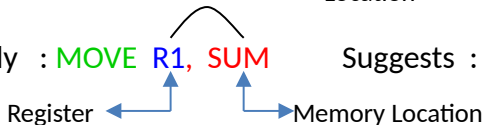
Suggests to copy contents of memory location **M** to register **R0**

- The corresponding instruction is



- Similarly : **MOVE R1, SUM**

Suggests :  $SUM \leftarrow [R1]$



- The instruction : **MOVE B, LOC** Means :  $LOC \leftarrow [B]$

# Register Transfer Notation

- Similar operations that involve **Registers only** are:

**R1**  $\leftarrow$  [**R2**]                      **MOVE R2, R1**

- **R1**  $\leftarrow$  [**R1**] + [**R3**]                      **ADD R3, R1**

Suggest to add contents of register R1 and register R3 and rewrite the R1 contents to store the sum.

- All such notations with leftward arrow ( $\leftarrow$ ) assignment operations involving register locations give rise to what is known as **Register Transfer Notation**

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 2. Assembly Language Notation T2:Ch2 2.4

Department of Computer Science and Engineering

# Assembly Language Notation

- An assignment statement in a High Level Language Program:

$$S = P + Q$$

- Here **P**, **Q** & **S** are variables.

- **Variable name** refers to *symbolic address of memory location* from which data operand can be read or written.

$$S \leftarrow [P] + [Q]$$

- This operation suggests contents of two memory locations **P** and **Q** are fetched from memory and transferred into processor where sum of two numbers is performed. The resulting sum is then sent back to memory and stored in memory location **S**.

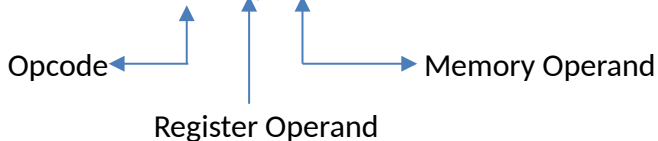
- An instruction to this effect in Assembly Language Notation:

**Add P, Q, S**

# Assembly Language Notation

- Consider  $R1 \leftarrow [R1] + [R3]$
- Equivalent Assembly Language instruction is **Add R3, R1**.
- Here the operation Code Mnemonic is **Add**
- Register Locations **R1, R3** represent *operand fields*

- Similarly, in the instruction **ADD R1, SUM**



- i.e.  $SUM \leftarrow [SUM] + [R1]$

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 3. Basic Instruction Types T2:Ch2 2.4

Department of Computer Science and Engineering

# Basic Instruction Types

- ☐ Three - Address Instruction
- ☐ Two - Address Instruction
- ☐ One - Address Instruction
- ☐ Zero - Address Instruction

## ➤ Three - Address Instruction

Suppose we would like to use a single machine instruction to perform the following addition operation:

$$S \leftarrow [P] + [Q]$$

We will have to use a **three - address instruction** to carry out addition and to store the sum in a third variable **S**.

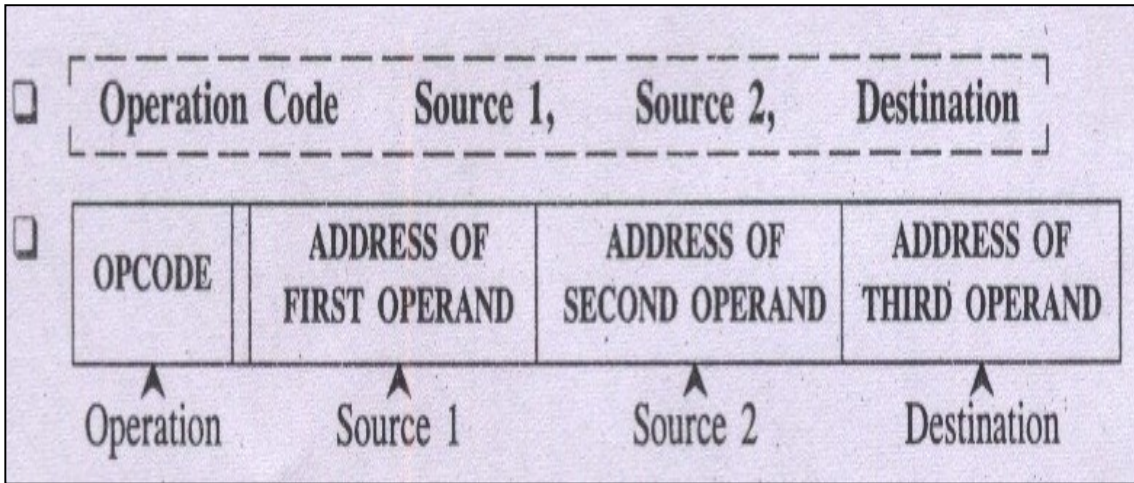
➤ Then three address instruction to perform addition is:

Add P, Q, S



# Basic Instruction Types

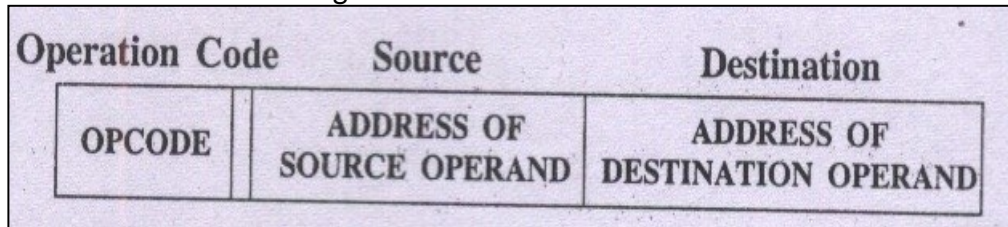
## General form of three – address instruction



# Basic Instruction Types

## Two – Address Instruction

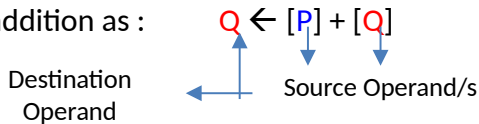
- Two – address instruction general format:



- Example of two – address instruction

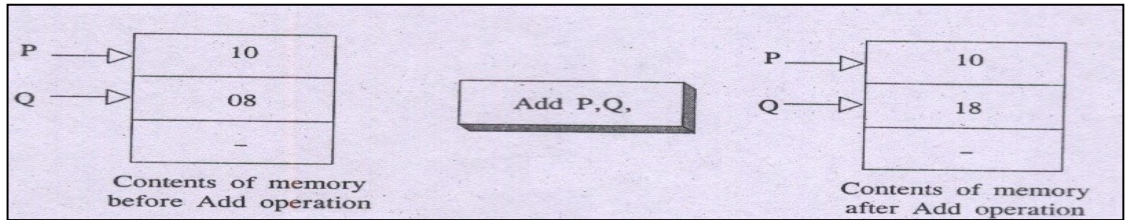
Add P, Q

- To perform operation of addition as :



# Basic Instruction Types

- Here one of the operand i.e. destination operand Q acts as source as well as destination.



- To perform  $S \leftarrow [P] + [Q]$

- Use two - address instructions sequence:  
`MOVE Q, S`  
`ADD P, S`

# Basic Instruction Types

## One-Address Instruction

- One -address instruction format specify a single operand along with operation code.
- The requirement of second operand is fulfilled by an implicit processor register known as **Accumulator (AC)**.

- Example of one-address instruction:

- It Specify:

- **Load P** Suggest  $AC \leftarrow [P]$

(Fetch P into AC)

- **Store S** Indicate  $S \leftarrow [AC]$

(Write AC into S)

- To perform  $S \leftarrow [P] + [Q]$

- Use one - address instructions : **Load P;**

$AC \leftarrow [P]$

**Add Q;**

$AC \leftarrow [AC] + [Q]$

**Store S;**

$S \leftarrow [AC]$

# Basic Instruction Types

## Zero – Address Instruction

- Stack – organized computer make use of a **special memory structure called push down stack to store operands**. In such computer machines it is possible to use instructions that contain only operation codes and **no explicit operands**.
- The name **Zero – address** specifies the absence of an address field of operands in machine instructions.
- Example of Zero – address instruction: **ADD**
- To perform operation of addition as **TOS  $\leftarrow$  (P + Q)**

# Basic Instruction Types

---

Mov A,Ri is equivalent  
Load A,Ri

and

Move Ri,A is equivalent to  
Store Ri,A

# Basic Instruction Types

➤ If processor supports ALU operations where one data may be in memory and other in register then the instruction sequence is for

ADD A,B,C :

In processors where arithmetic operations are allowed only on operands that are in processor registers, the  $C = A + B$  task can be performed by the instruction sequence

Move A,R<sub>i</sub>

Move B,R<sub>j</sub>

Add R<sub>i</sub>,R<sub>j</sub>

Move R<sub>j</sub>,C

In processors where one operand may be in the memory but the other must be in a register, an instruction sequence for the required task would be

Move A,R<sub>i</sub>

Add B,R<sub>i</sub>

Move R<sub>i</sub>,C

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 4. Instruction Execution And Straight-Line Sequencing

T2:Ch2 2.4

Department of Computer Science and Engineering



# Instruction Execution And Straight-Line Sequencing

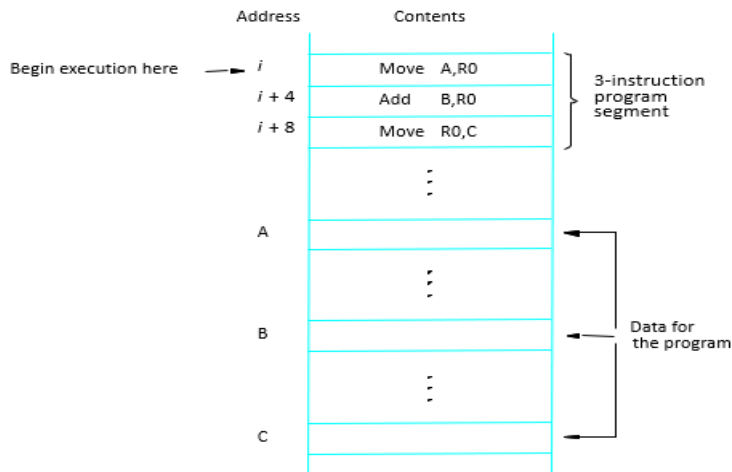


Figure 2.8. A program for  $C \leftarrow [A] + [B]$ .

## Assumptions:

- One memory operand per instruction
- 32-bit word length
- Memory is byte addressable
- Full memory address can be directly specified in a single-word instruction

## Two-phase procedure

- Instruction fetch
- Instruction execute

# Instruction Execution And Straight-Line Sequencing

- **PC – Program counter**: hold the address of the next instruction to be executed
- **Straight line sequencing**: If fetching and executing of instructions is carried out one by one from successive addresses of memory, it is called straight line sequencing.
- Major two phase of instruction execution
  - ❖ **Instruction fetch phase**: Instruction is fetched from memory and is placed in instruction register IR
  - ❖ **Instruction execute phase**: Contents of IR is decoded and processor carries out the operation either by reading data from memory or registers.

# Instruction Execution And Straight-Line Sequencing

|              |      |             |
|--------------|------|-------------|
| $i$          | Move | NUM1,R0     |
| $i + 4$      | Add  | NUM2,R0     |
| $i + 8$      | Add  | NUM3,R0     |
|              |      | ⋮           |
| $i + 4n - 4$ | Add  | NUM $n$ ,R0 |
| $i + 4n$     | Move | R0,SUM      |
|              |      | ⋮           |
| SUM          |      |             |
| NUM1         |      |             |
| NUM2         |      |             |
|              |      | ⋮           |
| NUM $n$      |      |             |

Figure 2.9. A straight-line program for adding  $n$  numbers.

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 5. Branching T2:Ch2 2.4

Department of Computer Science and Engineering

- Branch instructions are those which change the normal sequence of execution.
- Sequence can be changed either conditionally or unconditionally.
- Accordingly we have **conditional** branch instructions and **unconditional branch instruction**.
- Conditional branch instruction changes the sequence only when certain conditions are met.
- Unconditional branch instruction changes the sequence of execution irrespective of the condition of the results.

- Consider branch instruction in a program:

**Branch > 0 Loop**

- Program execution continues in a straight line sequence until encountering of **Branch > 0 Loop** instruction
- Execution results in a jump to location of **Loop**.
- That is a branch instruction loads a new value in PC - the memory location address (loop) at which program control is to be transferred to start/resume execution.
- As a result processor is prohibited from fetching and executing next instruction in straight – line sequence.

# Branching

Branch target

Conditional branch

Program loop

LOOP

Move N,R1

Clear R0

Determine address of  
"Next" number and add  
"Next" number to R0

Decrement R1

Branch>0 LOOP

Move R0,SUM

•  
•  
•

SUM

N

$n$

NUM1

NUM2

•  
•  
•

NUM  $n$

Figure 2.10. Using a loop to add  $n$  numbers.

## Check whether number Positive or negative

Move a,R0

Branch >0 Even

MoveR0,R2

Even:

Move R0,R1



# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 6. Condition Codes

T2:Ch2 2.4

Department of Computer Science and Engineering

# Condition Codes

- The **data conditions or status**, after an arithmetic or logical operation are **indicated by setting or clearing the flip - flops called *flags* or *condition codes***.
- Each flip-flop holding a data condition code is a one -bit storage cell (logic circuit) that can be set to a 1 or reset to 0 value.
- These condition codes are set/reset as a result of arithmetic and logical operations in the ALU.
- Condition code bits or flag bits are accommodated in a groups of 4 - bit, 8 bit or 16 - bit flag register or a status register in CPU.

# Condition Codes

---

- Results of various instructions are stored for subsequent use by conditional instructions
- This is done by recording the required info in individual bits, called as condition code flags – grouped together in special processor register called as **condition code register or status register**

# Condition Codes

## ➤ CONDITIONAL CODE FLAGS:

|              |                                                                |
|--------------|----------------------------------------------------------------|
| N – Negative | 1 if results are Negative<br>0 if results are Positive         |
| Z – Zero     | 1 if results are Zero<br>0 if results are Non zero             |
| V – Overflow | 1 if arithmetic overflow occurs<br>0 no overflow occurs        |
| C – Carry    | 1 if carry from MSB bit<br>0 if there is no carry from MSB bit |

Branch>0. Branch  
is taken when  
neither N or Z=1

# Condition Codes

➤ **Pentium processor** makes use of following condition codes:

- C** (carry flag)
- P** (parity flag)
- A** (Auxiliary carry flag)
- Z** (zero flag)
- S** (sign flag)
- O** (over flow flag)

# Condition Codes

---

List the steps needed to execute the machine instruction Add LOC, R0 in terms of transfers between memory and processor and some simple control commands. Assume that the instruction itself is stored in the memory at location INSTR and that this address is initially in register PC.

# Condition Codes

- Transfer the contents of register PC to register MAR
- Issue a Read command to memory, and then wait until it has transferred the requested word into register MDR
- Transfer the instruction from MDR into IR and decode it
- Transfer the address LOCA from IR to MAR
- Issue a Read command and wait until MDR is loaded
- Transfer contents of MDR to the ALU
- Transfer contents of R0 to the ALU
- Perform addition of the two operands in the ALU and transfer result into R0
- Transfer contents of PC to ALU
- Add 1 to operand in ALU and transfer incremented address to PC

# Condition Codes

---

Repeat the problem for machine instruction  
Add R1,R2,R3



# Condition Codes

- Transfer the contents of register PC to register MAR
- Issue a Read command to memory, and then wait until it has transferred the requested word into register MDR
- Transfer the instruction from MDR into IR and decode it
- Transfer contents of R1 and R2 to the ALU
- Perform addition of two operands in the ALU and transfer answer into R3
- Transfer contents of PC to ALU
- Add 1 to operand in ALU and transfer incremented address to PC

# Condition Codes

Give a short sequence of machine instructions for the task: “Add the contents of memory location A to those of location B, and place the answer in location C.”

Instructions                      Load LOC,  $R_i$

and                                Store  $R_i$ , LOC

are the only instructions available to transfer data between the memory and general purpose register  $R_i$ . Do not destroy the contents of either location A or B.

# Condition Codes

---

Load A,R0  
Load B,R1  
Add R0,R1  
Store R1,C

# Condition Codes

---

Suppose that move and add instructions are available in the format  
Move/Add loc1,loc2

Loci can be either memory or reg. Is it possible to use fewer instructions?  
Give the seq

# Condition Codes

---

Move B,C

Add A,C

# DIGITAL DESIGN AND COMPUTER ORGANIZATION

---

## 7. Generating Memory Addresses T2:Ch2 2.4

Department of Computer Science and Engineering

# Generating Memory Addresses

- The instruction block at **LOOP** needs to add a different number from a list during each iteration.
- The **Add** instruction must refer to a different memory address during each pass through the loop.
- The memory operand address cannot be directly given in a single **Add** instruction because it would require modification on each pass.
- A possible solution is to use a processor register, such as **Ri**, to hold the memory address.
- **Ri** can be initialized with the address **NUM1** before the loop and incremented by 4 after each iteration to point to the next memory address.
- This situation highlights the need for flexible ways to specify operand addresses.
- These methods are known as **addressing modes**, which vary in implementation but share common underlying concepts across different computers.

# Think about it

---

- Write assembly code in three two and one address format for SUB A,B,C
- Write assembly code in three two and one address format for  $A=B+C-D$



A processor executes instructions in **straight-line sequencing** unless:

- a) A Load instruction is encountered
- b) A Store instruction is encountered
- c) A Branch instruction is encountered
- d) A Condition Code is set

Which condition flag is set to 1 if an arithmetic operation results in a value too large for the number of bits available?

- a) Zero (Z)
- b) Carry (C)
- c) Overflow (V)
- d) Sign (N)

A processor executes instructions in **straight-line sequencing** unless:

- a) A Load instruction is encountered
- b) A Store instruction is encountered
- c) A Branch instruction is encountered
- d) A Condition Code is set

**Answer:** c) A Branch instruction is encountered

**Explanation:** Branch instructions alter the normal program counter (PC) sequence

Which condition flag is set to 1 if an arithmetic operation results in a value too large for the number of bits available?

- a) Zero (Z)
- b) Carry (C)
- c) Overflow (V)
- d) Sign (N)

**Answer:** c) Overflow (V)

**Explanation:** Overflow flag is set when the signed result exceeds representable range

- Which one of the following instruction formats uses the **accumulator (AC)** implicitly as one operand?  
a) Zero-address instruction   b) One-address instruction   c) Two-address instruction   d) Three-address instruction
- Consider the sequence of instructions:  
Load P  
Add Q  
Store S  
This corresponds to which instruction format?  
a) Three-address  
b) Two-address  
c) One-address  
d) Zero-address

- Which one of the following instruction formats uses the **accumulator (AC)** implicitly as one operand?
  - a) Zero-address instruction
  - b) One-address instruction
  - c) Two-address instruction
  - d) Three-address instruction

Answer: b) One-address instruction

Explanation: In one-address instructions, one operand is explicitly given, while the accumulator (AC) is the implicit second operand

- Consider the sequence of instructions:

Load P

Add Q

Store S

This corresponds to which instruction format?

- a) Three-address
- b) Two-address
- c) One-address
- d) Zero-address

Answer: b) One-address instruction



# THANK YOU

---

**Team DDCO**

Department of Computer Science